



互连网络

- ◆ 基本概念
- ◆ 互连网络种类
- ◆ 消息传递机制



基本概念

- ◆ 互连网络的作用
- ◆ 互连网络的表示
- ◆ 常用互连函数
- ◆ 互连网络的结构参数
- ◆ 互连网络的性能参数



互连网络的作用

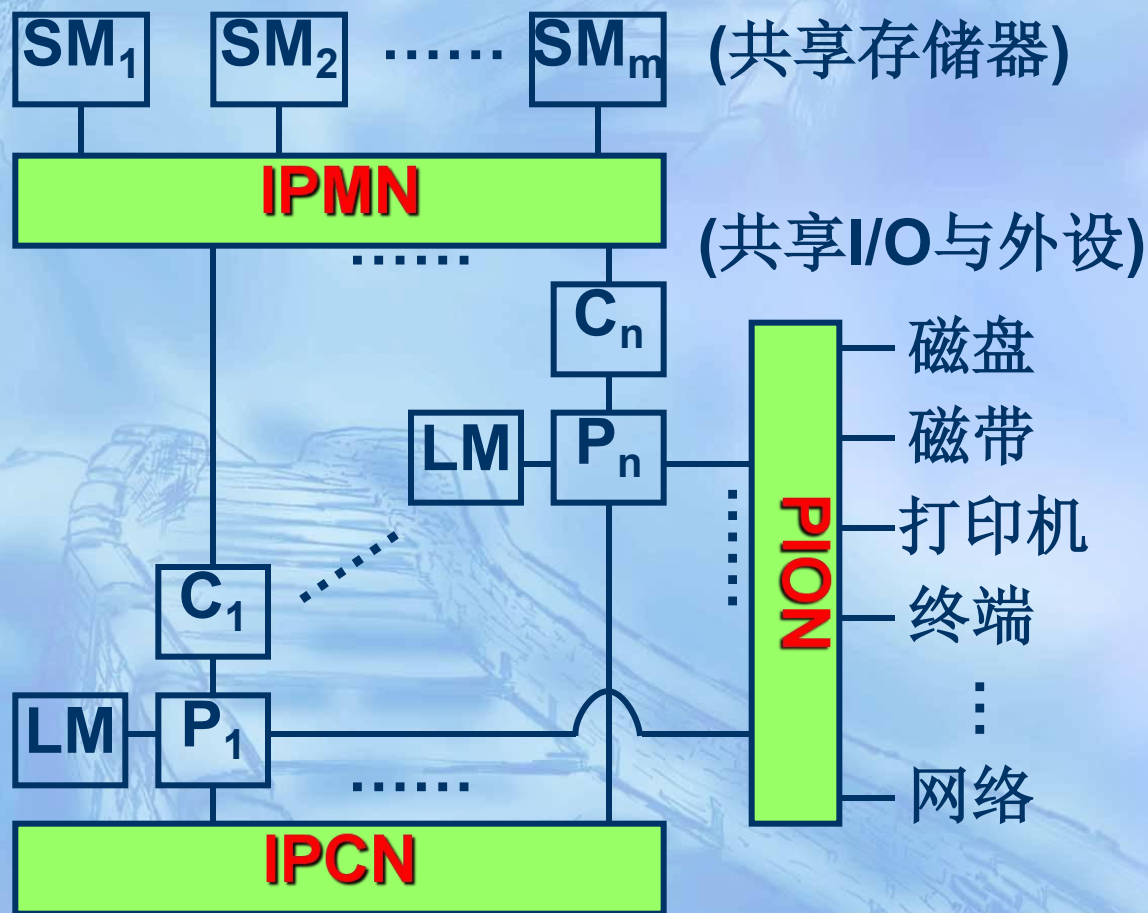
互连网络是一种由**开关元件**按照一定的**拓扑结构**和**控制方式**构成的网络，用于实现计算机系统内部多个处理机或多个功能部件之间的相互连接。互连网络已成为并行处理系统的核心组成部分。互连网络对整个计算机系统的性能价格比有着决定性的影响。



举例说明

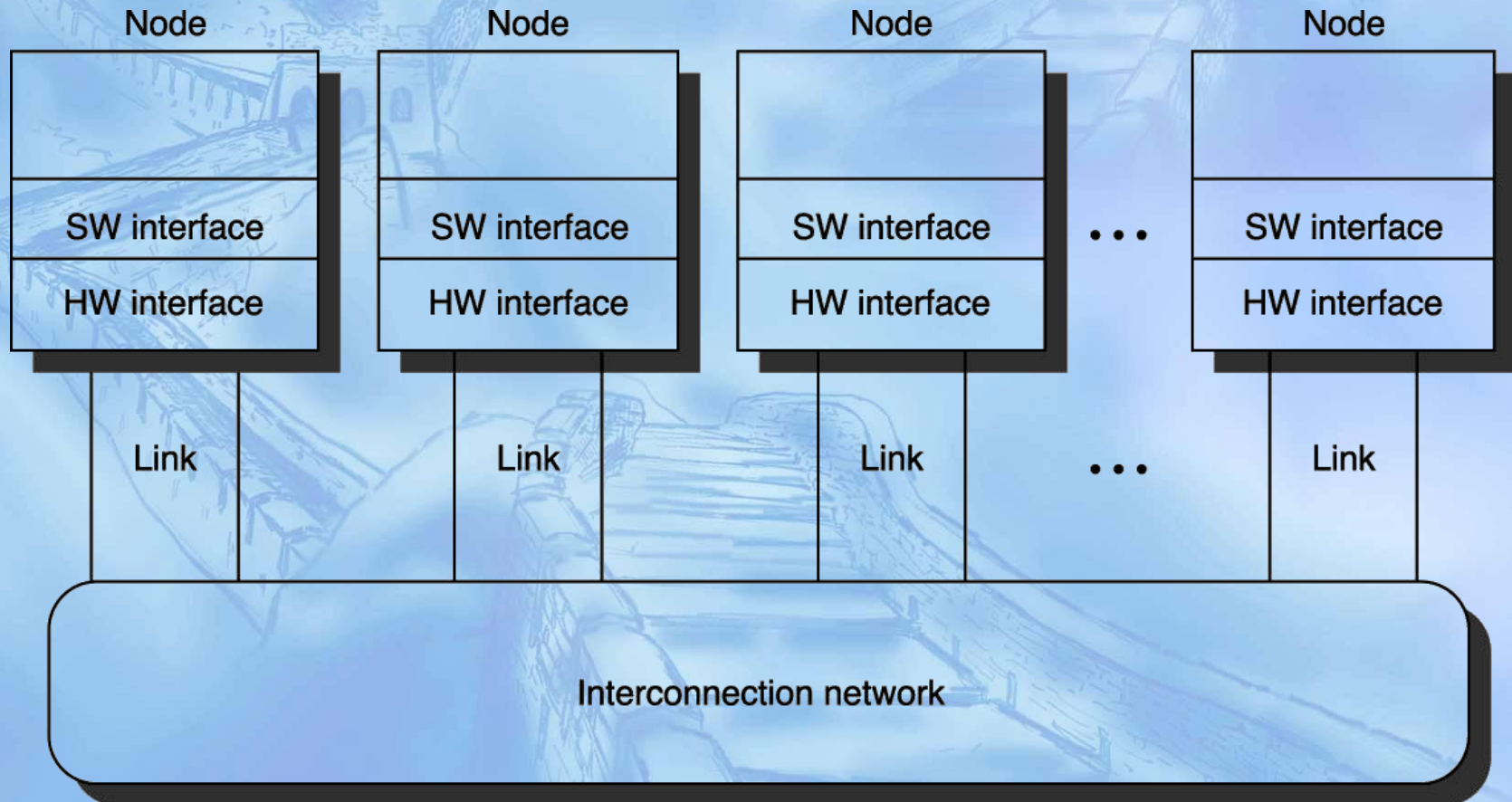
(多处理机)

SM : 共享存储器
LM : 本地存储器
P : 处理机
C : 高速缓存
IPMN: 内部处理机-存储器网络
IPCN: 内部处理机间通信网络
PION: 处理机-输入输出间网络





基本模型





互连网络的表示

◆ 互连函数表示法

自变量和函数常用二进制表示。

例如： $f(x_{n-1}x_{n-2}\dots x_1x_0) = x_0x_{n-2}\dots x_1x_{n-1}$ 。

◆ 输入输出对应表示法



输入: 0	1	2	...	n
输出: f(0)	f(1)	f(2)	...	f(n)

输入: 0	1	2	3	4	5	6	7
输出: 0	4	2	6	1	5	3	7

◆ 循环表示法（有时）

$(x_0 x_1 \dots x_{j-1})$ 表示 $f(x_0)=x_1, f(x_1)=x_2, \dots, f(x_{j-1})=x_0$

j为循环长度



常用互连函数

- ◆ 恒等置换
- ◆ 交换置换
- ◆ 方体置换
- ◆ 均匀洗牌置换

- ◆ 蝶式置换
- ◆ 位序颠倒置换
- ◆ 移数置换
- ◆ 加减 2^i 置换



恒等置换

$$I(x_{n-1}x_{n-2}\dots x_1x_0) = x_{n-1}x_{n-2}\dots x_1x_0$$

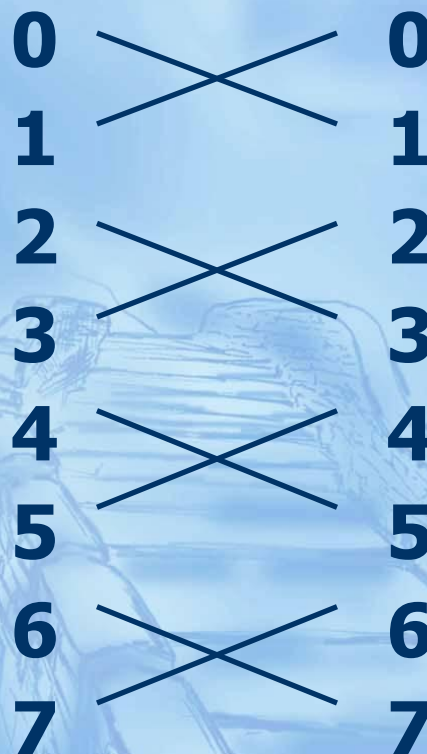
0	——	0
1	——	1
2	——	2
3	——	3
4	——	4
5	——	5
6	——	6
7	——	7

N=8 的恒等置换



交换置换

$$E(x_{n-1}x_{n-2}\dots x_1x_0) = x_{n-1}x_{n-2}\dots x_1\bar{x}_0$$



N=8 的交换置换



方体置换

— 互连函数

$$C_k(x_{n-1}x_{n-2}\dots x_k\dots x_1x_0) = x_{n-1}x_{n-2}\dots \bar{x}_k\dots x_1x_0$$

例如：当N=8时，有3种函数，每种能表示8个节点之间的连接关系。

$$C_2(x_2x_1x_0) = \bar{x}_2x_1x_0$$

$$C_1(x_2x_1x_0) = x_2\bar{x}_1x_0$$

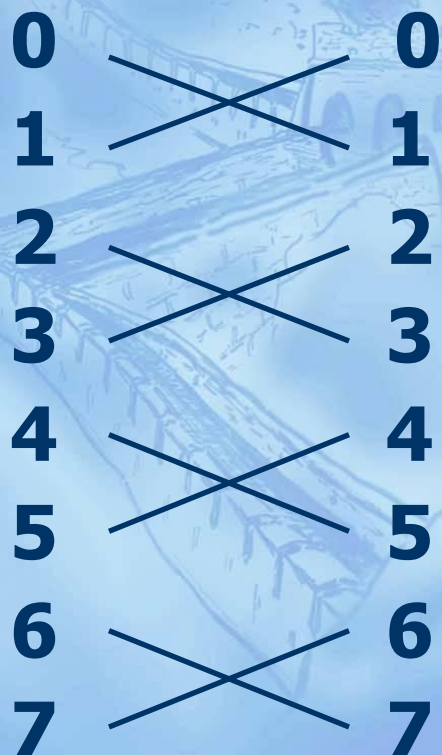
$$C_0(x_2x_1x_0) = x_2x_1\bar{x}_0$$

C_0 就是交换置换。

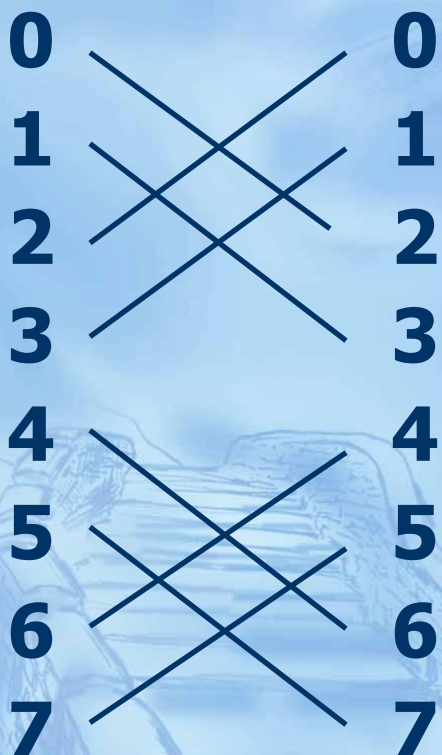


方体置换

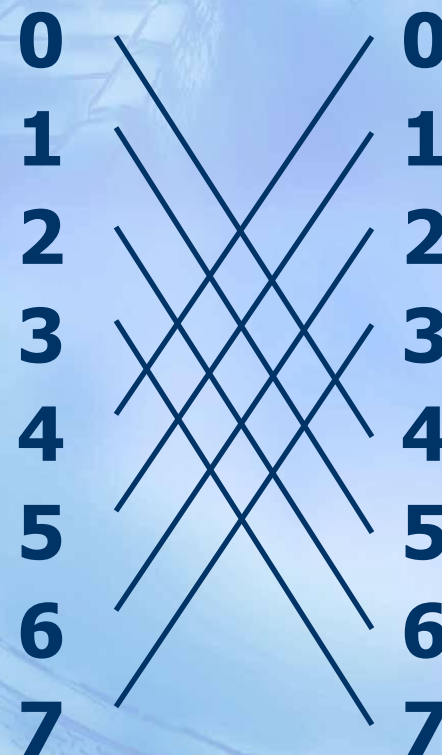
— 图示 (N=8)



C_0 方体置换



C_1 方体置换



C_2 方体置换



方体置换

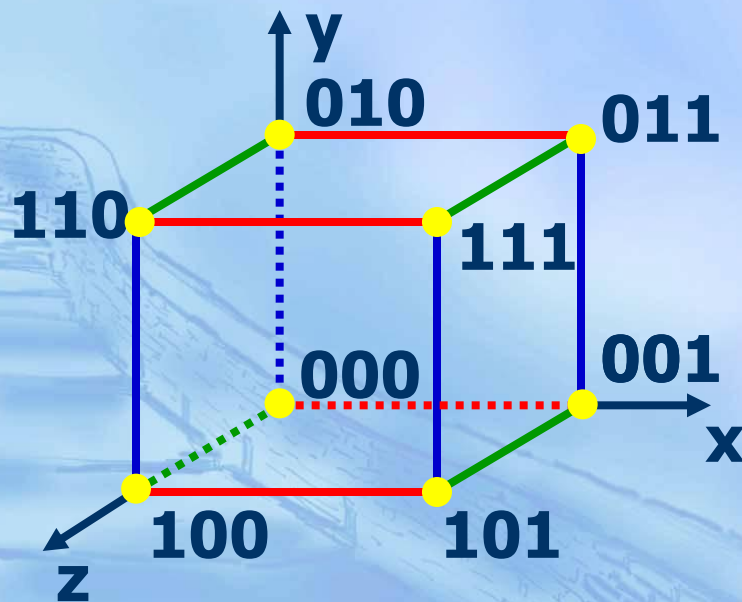
— 提示

由于方体置换函数主要用于超立方体互连网中，因此也称为**超立方体函数**，用**Cube**表示，如：**Cube₀**、**Cube₁**、**Cube₂**等。

$$\text{Cube}_0 = (b_2 b_1 \bar{b}_0)$$

$$\text{Cube}_1 = (b_2 \bar{b}_1 b_0)$$

$$\text{Cube}_2 = (\bar{b}_2 b_1 b_0)$$





均匀洗牌置换

— 互连函数

- ♦ 均匀洗牌(shuffle:循环左移一位)

$$\sigma(x_{n-1}x_{n-2}\dots x_k\dots x_1x_0) = x_{n-2}\dots x_k\dots x_1x_0x_{n-1}$$

- ♦ 子洗牌(subshuffle:最低k位循环左移一位)

$$\sigma_{(k)}(x_{n-1}x_{n-2}\dots x_kx_{k-1}x_{k-2}\dots x_1x_0) = \\ x_{n-1}x_{n-2}\dots x_kx_{k-2}\dots x_1x_0x_{k-1}$$

- ♦ 超洗牌(supershuffle:最高k位循环左移一位)

$$\sigma^{(k)}(x_{n-1}x_{n-2}\dots x_{n-k}x_{n-k-1}\dots x_1x_0) = \\ x_{n-2}\dots x_{n-k}x_{n-1}x_{n-k-1}\dots x_1x_0$$

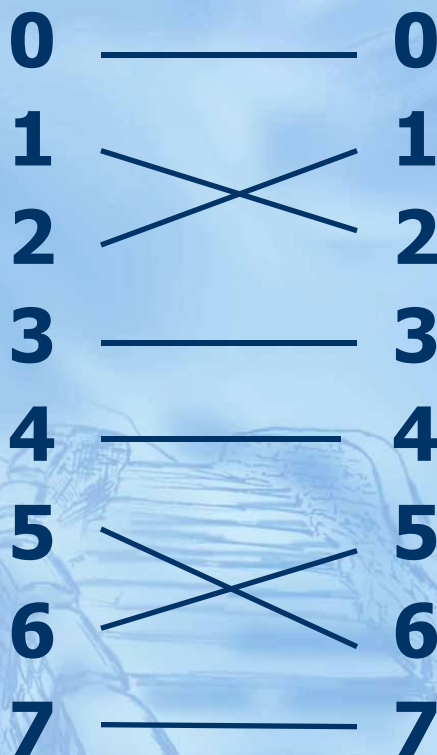


均匀洗牌置换

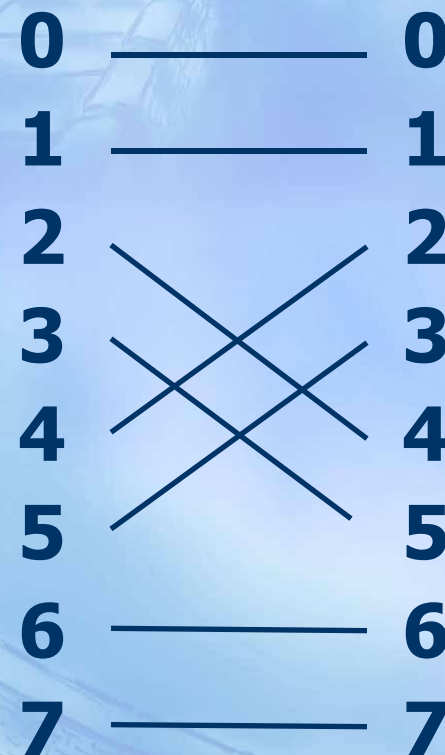
— 图示 (N=8)



均匀洗牌 σ



子洗牌 $\sigma_{(2)}$



超洗牌 $\sigma^{(2)}$



均匀洗牌置换

— 提示

◆ 三种置换之间的关系

$$\sigma^{(n)}(X) = \sigma_{(n)}(X) = \sigma(X)$$

$$\sigma^{(1)}(X) = \sigma_{(1)}(X) = X$$

◆ 逆洗牌（二进制节点号循环右移一位）

$$\sigma^{-1}(x_{n-1}x_{n-2}\dots x_1x_0) = x_0x_{n-1}x_{n-2}\dots x_1$$



均匀洗牌置换

— 应用

均匀洗牌与逆均匀洗牌是两个十分有用的互连函数，以它们代表的链路与以交换置换代表的开关多级组合起来可构成**Omega** (Ω) 网络与逆**Omega** (Ω) 网络。 σ 函数在实现多项式求值、矩阵转置和FFT等并行运算以及并行排序等方面都得到广泛的应用。



蝶式置换

— 互连函数

- ◆ 蝶式(butterfly:高低位互换)

$$\beta(x_{n-1}x_{n-2}\dots x_k\dots x_1x_0)=x_0x_{n-2}\dots x_k\dots x_1x_{n-1}$$

- ◆ 子蝶式(subbutterfly:最低k位高低位互换)

$$\beta_{(k)}(x_{n-1}x_{n-2}\dots x_kx_{k-1}x_{k-2}\dots x_1x_0)=$$

$$x_{n-1}x_{n-2}\dots x_kx_0x_{k-2}\dots x_1x_{k-1}$$

- ◆ 超蝶式(superbutterfly:最高k位高低位互换)

$$\beta^{(k)}(x_{n-1}x_{n-2}\dots x_{n-k+1}x_{n-k}x_{n-k-1}\dots x_1x_0)=$$

$$x_{n-k}x_{n-2}\dots x_{n-k+1}x_{n-1}x_{n-k-1}\dots x_1x_0$$

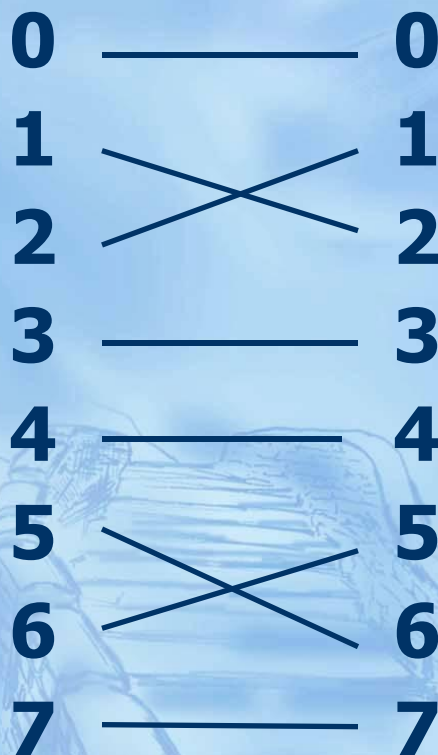


蝶式置换

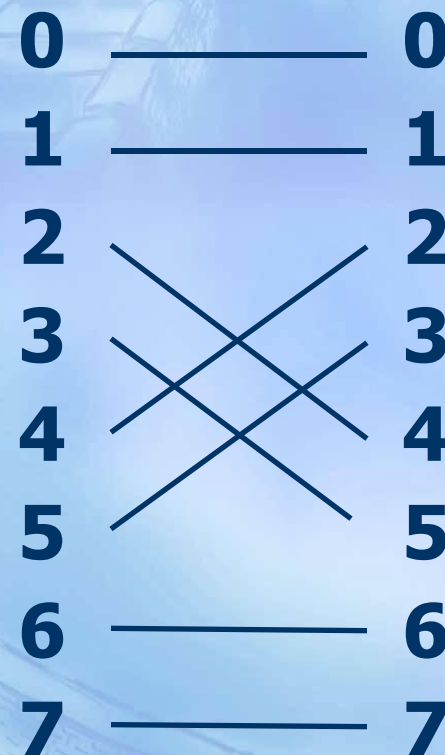
— 图示 (N=8)



蝶式 β



子蝶式 $\beta_{(2)}$



超蝶式 $\beta_{(2)}$



蝶式置换

— 提示

◆ 三种置换之间的关系

$$\beta^{(n)}(x) = \beta_{(n)}(x) = \beta(x)$$

$$\beta^{(1)}(x) = \beta_{(1)}(x) = x$$

◆ 应用

蝶式与子蝶式置换和交换置换多级组合可作为构成方体多级网络的基础。



位序颠倒置换

— 互连函数

◆ 位序颠倒置换 (Bit Reversal: 位序颠倒)

$$\rho(x_{n-1}x_{n-2}\dots x_k\dots x_1x_0) = x_0x_1\dots x_k\dots x_{n-2}x_{n-1}$$

◆ 子位序颠倒置换 (最低k位的位序颠倒)

$$\rho_{(k)}(x_{n-1}x_{n-2}\dots x_k \color{red}{x_{k-1}x_{k-2}\dots x_1x_0}) = \\ x_{n-1}x_{n-2}\dots x_k \color{red}{x_0x_1\dots x_{k-2}x_{k-1}}$$

◆ 超位序颠倒置换 (最高k位的位序颠倒)

$$\rho^{(k)}(\color{red}{x_{n-1}x_{n-2}\dots x_{n-k+1}x_{n-k}}x_{n-k-1}\dots x_1x_0) = \\ \color{red}{x_{n-k}x_{n-k+1}\dots x_{n-2}x_{n-1}}x_{n-k-1}\dots x_1x_0$$

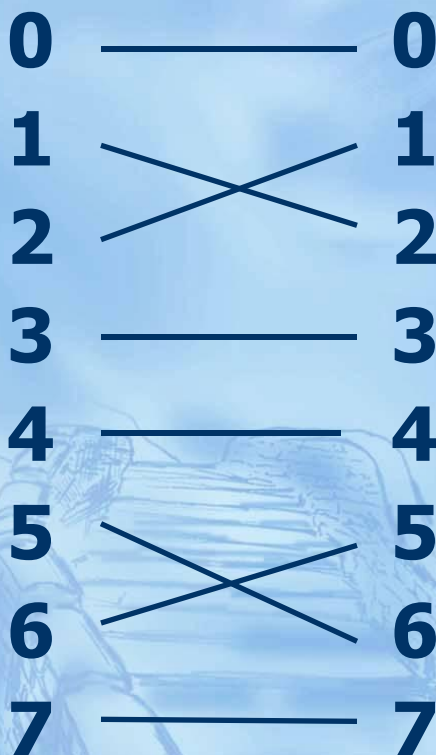


位序颠倒置换

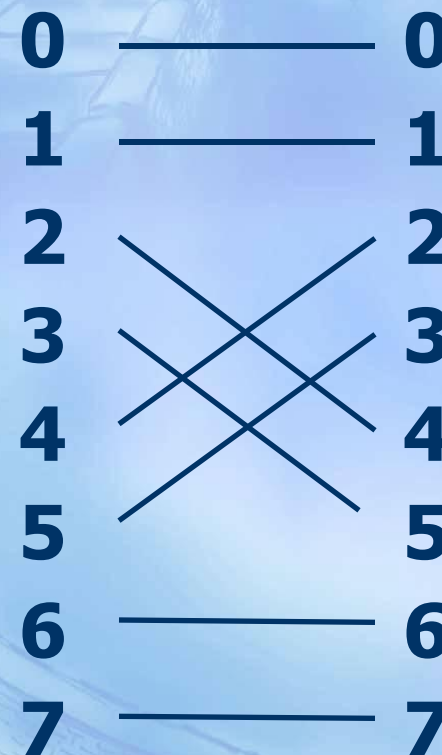
— 图示 (N=8)



位序颠倒 ρ



子位序颠倒 $\rho_{(2)}$



超位序颠倒 $\rho_{(2)}$



移数置换

— 互连函数

◆ 移数函数

$$a(x) = (x + k) \bmod N, \quad 0 \leq x \leq N-1, k \text{ 为移过的距离}$$

◆ 子移数函数

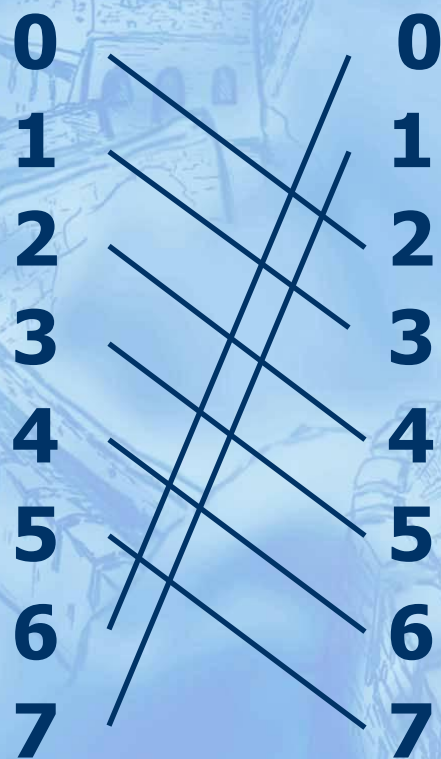
$$a(x)_{(n-1):r} = (x)_{(n-1):r}; \text{(高位保持不变)}$$

$$a(x)_{(r-1):0} = ((x_{(r-1):0} + k) \bmod 2^r); \text{(低位循环移动)}$$

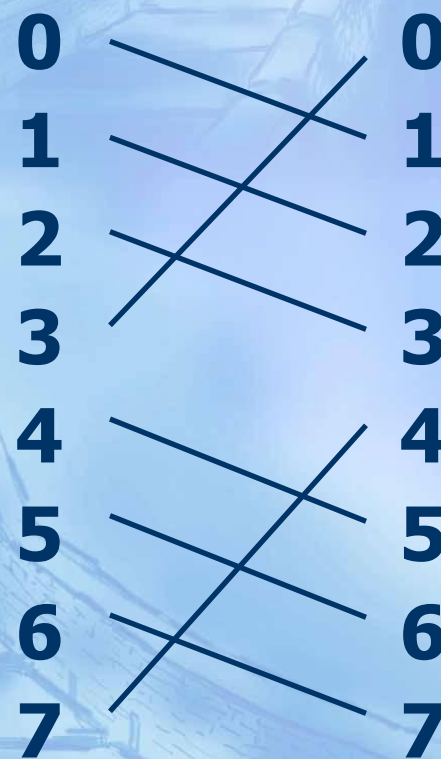


移数置换

— 图示 (N=8)



移数置换 $k=2$



子移数置换 ($k=1, r=2$)



加减 2^i 置换

实际上是一种移数置换。

$$\text{PM}_{2^+_i}(x) = (x + 2^i) \bmod N$$

$$\text{PM}_{2^-_i}(x) = (x - 2^i) \bmod N$$

其中： $0 \leq x \leq N-1$ ， $0 \leq i \leq n-1$ ， $n = \log_2 N$ 。



你掌握了吗？

假设16个处理机的编号分别为0、1、...、15，采用单级互连网络。互连函数分别为：

- (1) Cube_3
- (2) PM2_{+3}
- (3) PM2_{-0}
- (4) Shuffle
- (5) Butterfly
- (6) Reversal

问：第12号处理机分别与哪一个处理机相连？



你掌握了吗？

解： $(12)_{10} = (1100)_2$

(1) **Cube₃**

(2) **PM2₊₃**

(3) **PM2₋₀**

(4) **Shuffle**

(5) **Butterfly**

(6) **Reversal**

1100最高位取反得0100

4号处理机

$(12+2^3) \text{ MOD } 16 = 4$

4号处理机

$(12-2^0) \text{ MOD } 16 = 11$

11号处理机

1100循环左移1位得到1001

9号处理机

1100的最高最低位交换0101

5号处理机

1100的位序反过来为0011

3号处理机



互连网络的结构参数

互连网络通常是用有向边或无向边连接有限个节点的图来表示，主要结构参数有：

- ◆ **网络规模 N** ：网络中节点的个数。表示该网络所能连接的部件数量。
- ◆ **节点度 d** ：与节点相连接的边数（通道数）称为节点度，包括入度和出度。进入节点的边数叫**入度**，从节点出来的边数则叫**出度**。
- ◆ **节点距离**：对于网络中的任意两个节点，从一个节点出发到另一个节点终止所需要跨越的边数的最小值。
- ◆ **网络直径 D** ：网络中任意两个节点间距离的最大值。用节点间的连接边数表示。**网络直径应当尽可能地小。**



互连网络的结构参数

- ◆ **节点间的线长**：两个节点间连线的长度，用米等表示。
- ◆ **等分宽度b**：当网络被切成节点数相等的两半时，沿切口的最小边数（通道数）称为通道等分宽度。
 - **线等分宽度**： $B = b \times w$
其中： w 为通道宽度（用位表示）。
 - 该参数主要反映了网络最大流量。
- ◆ **对称性**：从任何节点看到拓扑结构都是一样的网络称为对称网络。对称网络比较易实现，编程也较容易。



互连网络的性能参数

◆ 时延

- **通信时延**：从源节点到目的节点传送一条消息所需的总时间，主要有：
 - **软件时延**：在源节点和目的节点用于收发信息的软件所需要的执行时间，主要取决于处理消息的软件内核；
 - **通道时延**：通过通道传送消息所化时间（消息长度/通道带宽），通常由瓶颈链路的通道带宽决定；
 - **选路时延**：消息在传送路径上所需的一系列选路决策所需的时间开销，与路径上的节点数成正比；
 - **竞争时延**：为避免/解决冲突所需的时间，很难预测



互连网络的性能参数

◆ 时延

- **网络时延**：通道时延和选路时延的总和，由网络硬件特征决定，与程序行为和网络传输状态无关

◆ 带宽

- **端口带宽**：单位时间内从该端口传送到其他端口的最大信息量
 - **对称网络**：端口带宽与端口位置无关，网络的端口带宽与各端口的端口带宽相同
 - **非对称网络**：端口带宽是指所有端口带宽的最小值



互连网络的性能参数

◆ 带宽

- **聚集带宽**：指网络从一半节点到另一半节点，单位时间内能够传送的最大信息量

例：**HPS**是一种对称网络，网络规模**N**的上限是**512**，端口带宽是**40MB/s**，因此**HPS**的聚集带宽是

$$(40\text{MB/s} * 512) / 2 = 10.24\text{GB/s}$$

- **等分带宽**：与等分宽度对应的切平面中，所有边合起来单位时间所能传送的最大信息量



互连网络种类

- ◆ 静态互连网络
- ◆ 动态互连网络



静态互连网络

在各节点之间有固定的连接通路，在运行过程中不能改变的网络结构。按拓朴结构又可分为一维、二维、三维等，一维的有线性阵列结构；二维的有环形、树形、星形、网格形等；三维的有立方体等；三维以上的有超立方体等。静态互连网络灵活性和适应性较差，很少使用。



线性阵列

- ◆ 有 N 个节点，节点度 $d=2$ （其中端节点的度 $d=1$ ），网络直径 $D=N-1$ ，等分宽度 $b=1$ ，拓扑结构不对称。
- ◆ **特点：**线性阵列结构最简单，实现成本低；但网络的延迟比较大， S_0 有信息发送到 S_{N-1} 必须通过所有其他节点。
- ◆ **应用：**当 N 较小（如 $N=2$ 或 3 ）时，实现线性阵列是相当经济的。



提示：线性阵列与总线的区别



环 形

◆ 单向环

右环网采用 $PM2_{+0}$ 函数，左环网采用 $PM2_{-0}$ 函数，对称，直径 $D=N-1$ ，节点度 $d=2$ 。

◆ 双向环

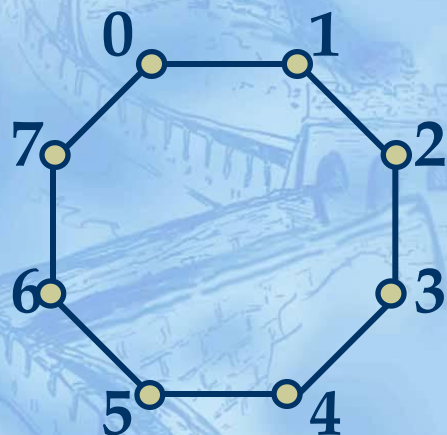
又称一维邻居网，采用 $\{PM2_{+0}, PM2_{-0}\}$ 函数，对称，直径 $D=N/2$ ，节点度 $d=2$ 。

◆ 弦环网

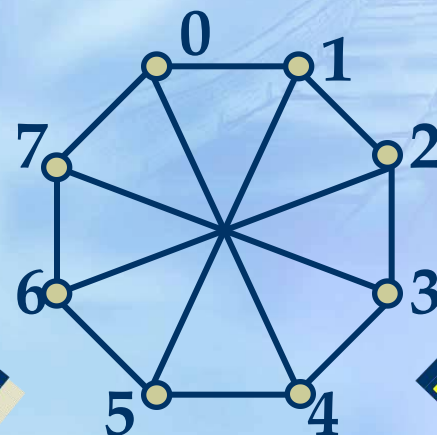
将节点度 d 由2提高至 ≥ 3 。增加的弦愈多，则节点度 d 愈高，网络直径 D 愈小。极端情况是全连接，网络直径 $D=1$ 。



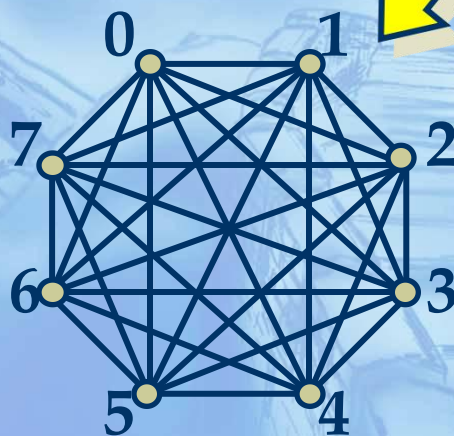
环 形



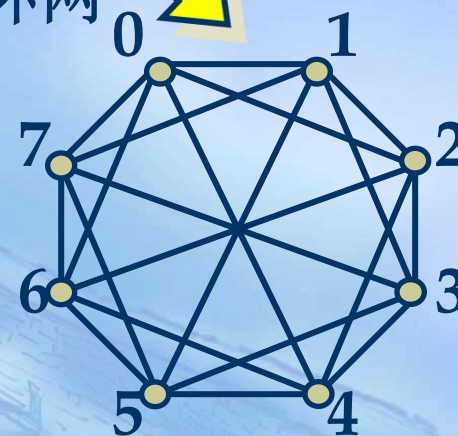
环形网



度 $d=3$ 的弦环网



全连接



循环移数网



循环移数网络

循环移数网络是将环上每个节点与其距离为**2**的整数幂的节点之间连接构成。即，采用 $2n-1$ 个互连函数： $PM_{2^{\pm i}}(j)=(j \pm 2^i) \bmod N$ ， $n=\log_2 N$ ， $0 \leq i \leq n-1$ ， $0 \leq j \leq N-1$ ；其中： $PM_{2^{+(n-1)}}=PM_{2^{-(n-1)}}$ 。

若循环移数网的网络规模 $N=2^n$ ，则节点度 $d=2n-1$ ，网络直径 $D=n/2$ 。例如：节点数 $N=64$ ， $n=6$ ， $d=11$ ， $D=3$ 。



树 形

◆ **二叉树**：一棵k层完全平衡的二叉树有 $N=2^k-1$ 个节点，节点度 $d=3$ ，直径 $D=2(k-1)$ ，等分宽度 $b=1$

特点：结构可扩展，但直径比较长

应用：**DADO**多处理机采用10层二叉树互连1023个节点

◆ **星形**：一种特殊的2层树，节点度很高，为 $d=N-1$ ，直径 $D=2$ ，等分宽度 $b=\lfloor N/2 \rfloor$ 。

特点：可靠性比较差，只要中心节点出故障，整个系统就会瘫痪。

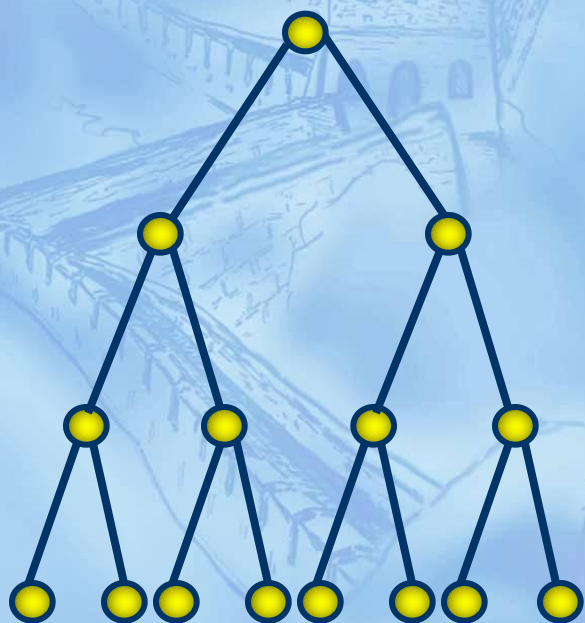
应用：一般用于有集中监督节点的系统中

◆ **二叉胖树**：缓解了根节点通信速度高的矛盾。

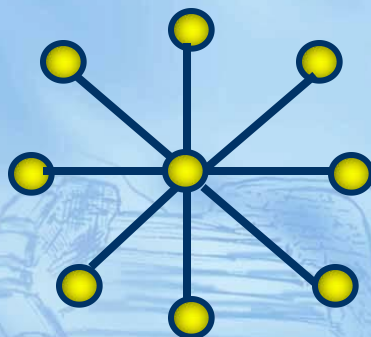
应用：在**CM-5**计算机中得到了应用



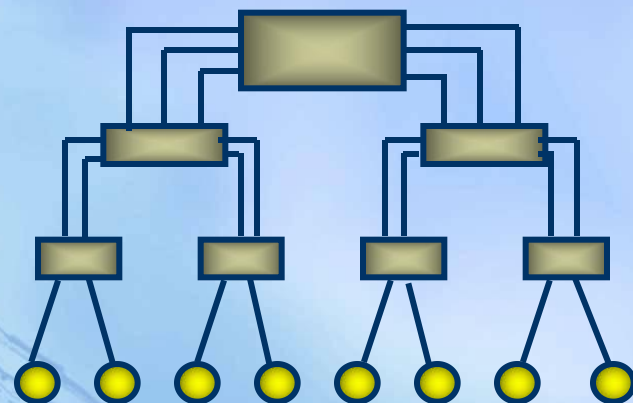
树形图示



二叉树网



星形网



二叉胖树网



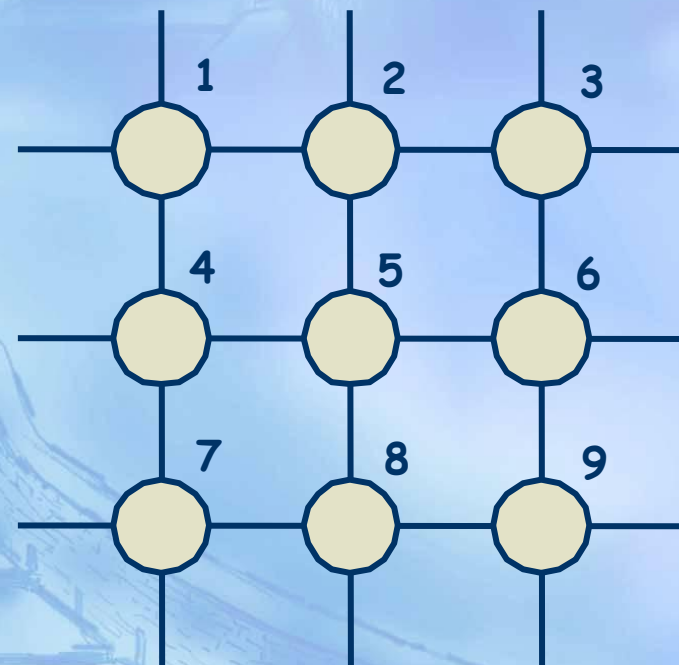
网格形

网格形是一种比较流行的网络结构，
有各种变体形式在ILLIAC IV、MPP、DAP、
CM-2和Intel Paragon等中得到了实现。



二维网格

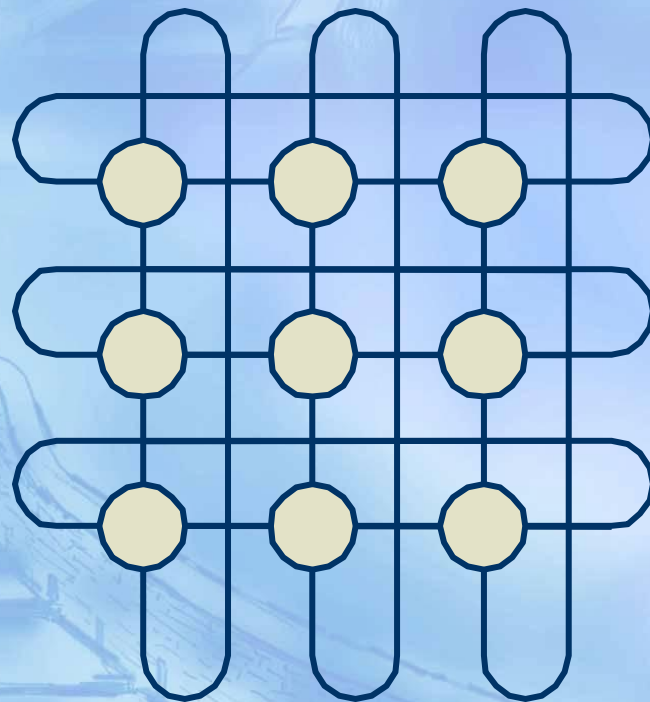
- ◆ 一般的二维网格有 $N = n_1 \times n_2$ 个节点组成，直径是 $D = (n_1 - 1) + (n_2 - 1)$ 。实际上经常是 $n_1 = n_2 = n$ ，此时网络直径 $D = 2(n - 1)$ ，节点度 $d = 4$ ，等分宽度 $b = n$ ，是一种非对称的拓扑结构。
- ◆ 一个由 $N = n^k$ 个结点构成的 k 维网格形网络（每维 n 个结点）的内部结点度 $d = 2k$ ，网络直径 $D = k(n - 1)$ 。





环形二维网格

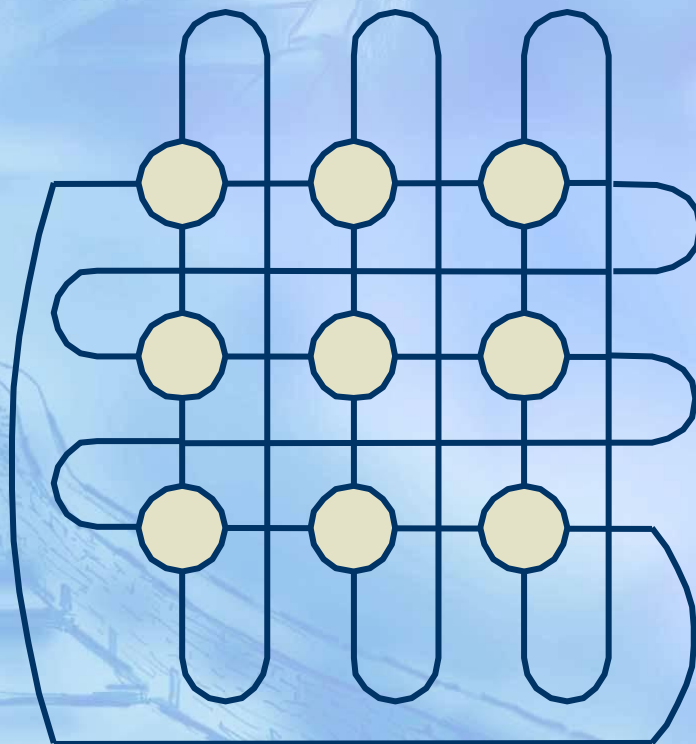
环形二维网格沿阵列每行每列都有环形连接，将环形和网格形组合在一起，并能向高维扩展。 $n \times n$ 二元环网的节点度 $d=4$ ，直径为 $D=2 \times \lfloor n/2 \rfloor$ ，等分宽度 $b=2n$ 。环网是一种对称的拓扑结构。





ILLIAC网格

二维网格逐行（或列）串形连接。 $n \times n$ ILLIAC 网格的直径为 $D=n-1$ ，为纯网格直径的一半；等分宽度 $b=2n$ 。例如：Illiac IV 计算机的 8×8 Illiac 网格，其节点度 $d=4$ ，直径 $D=7$ ，等分宽度 $b=16$ 。





ILLIAC网格

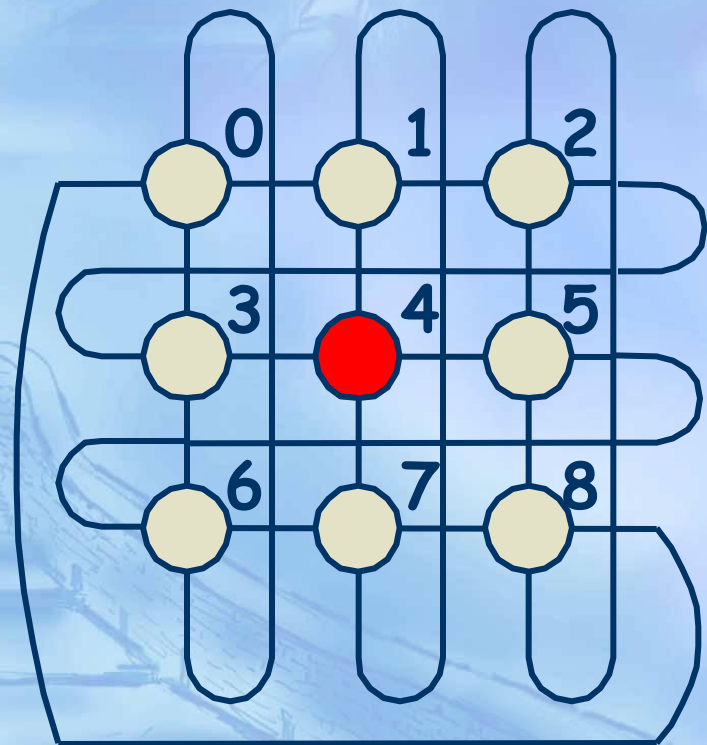
ILLIAC网络的节点
连接按一定的算法进行:

$$\text{Illiac}_{+1}(j) = (j+1) \bmod(N)$$

$$\text{Illiac}_{-1}(j) = (j-1) \bmod(N)$$

$$\text{Illiac}_{+n}(j) = (j+n) \bmod(N)$$

$$\text{Illiac}_{-n}(j) = (j-n) \bmod(N)$$





超立方体

超立方体也称 **n 维立方体**（**二元 n -立方体**），由 $N = 2^n$ 个节点组成，分布在 n 维上，每维有两个节点。超立方体节点度 $d = n$ ，网络直径 $D = n$ ，等分宽度 $b = N/2$ ，网络结构是对称的。每个节点由 n 个二进制数实行编号，相邻节点只能有一个数字不同。

为实现一个 n 维立方体，只要把两个 $n-1$ 维立方体中相对应的结点用链路连接起来即可。共需要 2^{n-1} 条链路。

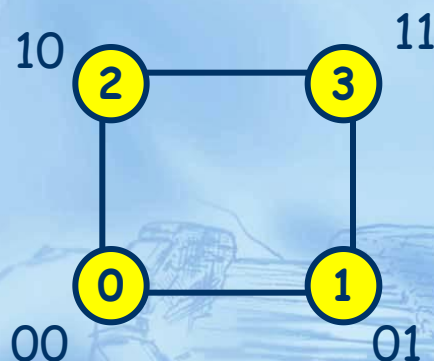
特点：超立方体缺乏可扩展性，难以组成高维超立方体（难以打包成模块化）。



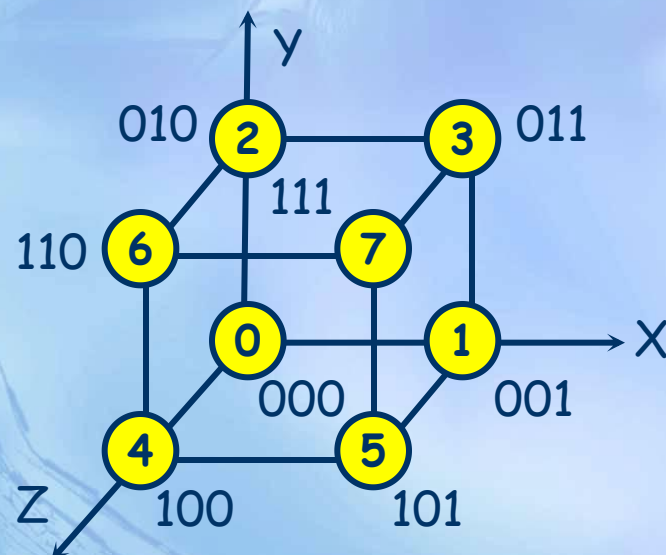
超立方体



1维立方体



2维立方体

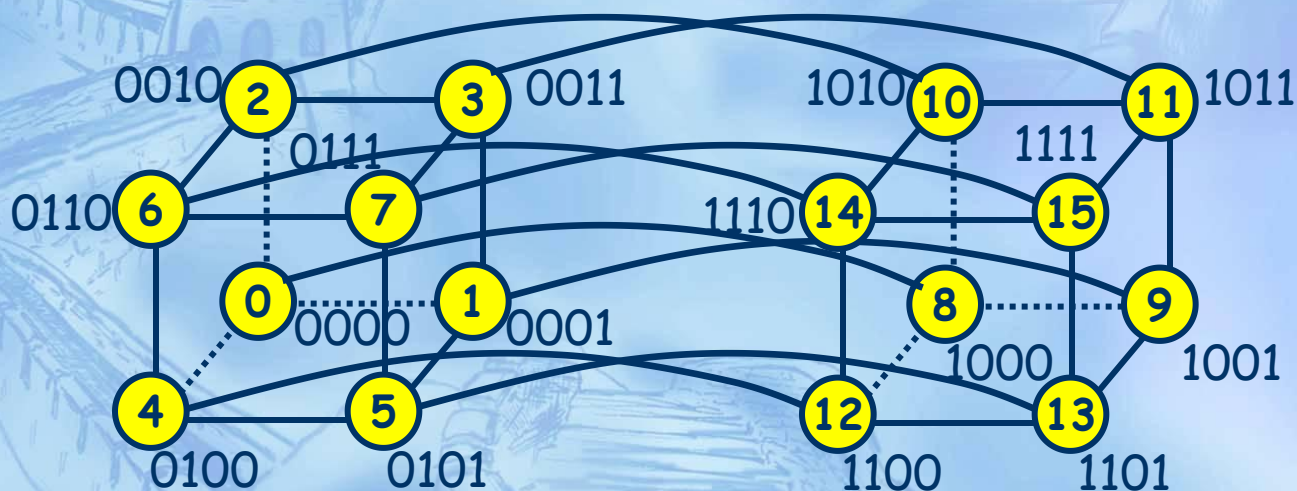


3维立方体



超立方体

4维立方体



在20世纪80年代，超立方体网络是一种非常流行的结构。Intel公司的iPSC/1、iPSC/2和nCUBE都采用了此结构。其他诸如二叉树和网格状等结构都可以嵌入超立方体。

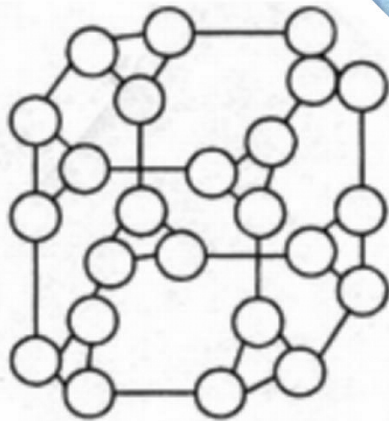


带环立方体

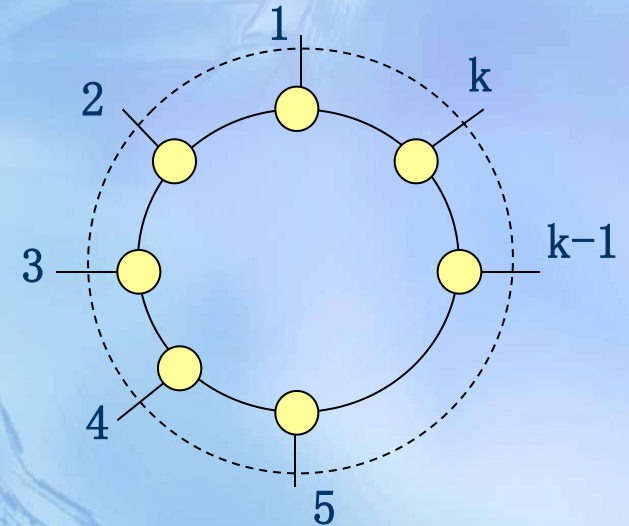
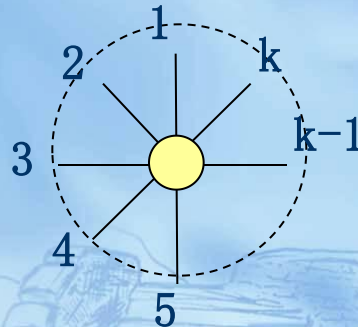
- ◆ **带环3-立方体 (3-CCC)**：把3维立方体的每个节点换成一个由3个节点构成的环而形成的。
- ◆ 高维化：**带环k-立方体**（简称**k-CCC**）
 - k维立方体的变形，它是通过用k个节点构成的环取代k维立方体中的每个节点而形成的。
 - 网络规模为 $N=k \times 2^k$
 - 网络直径为 $D=2k-1+\lfloor k/2 \rfloor$ ，比k维立方体的直径大一倍
 - 等分宽度为 $b=N/(2k)$
 - **节点度d=3**，与维数无关，因而具有很好的扩展性



带环立方体 (3-CCC)



(a) 带环3-立方体



(b) 将 k 维立方体的每个节点用由 k 个节点的环来代替，组成带环 k -立方体组成



k元n-立方体网络

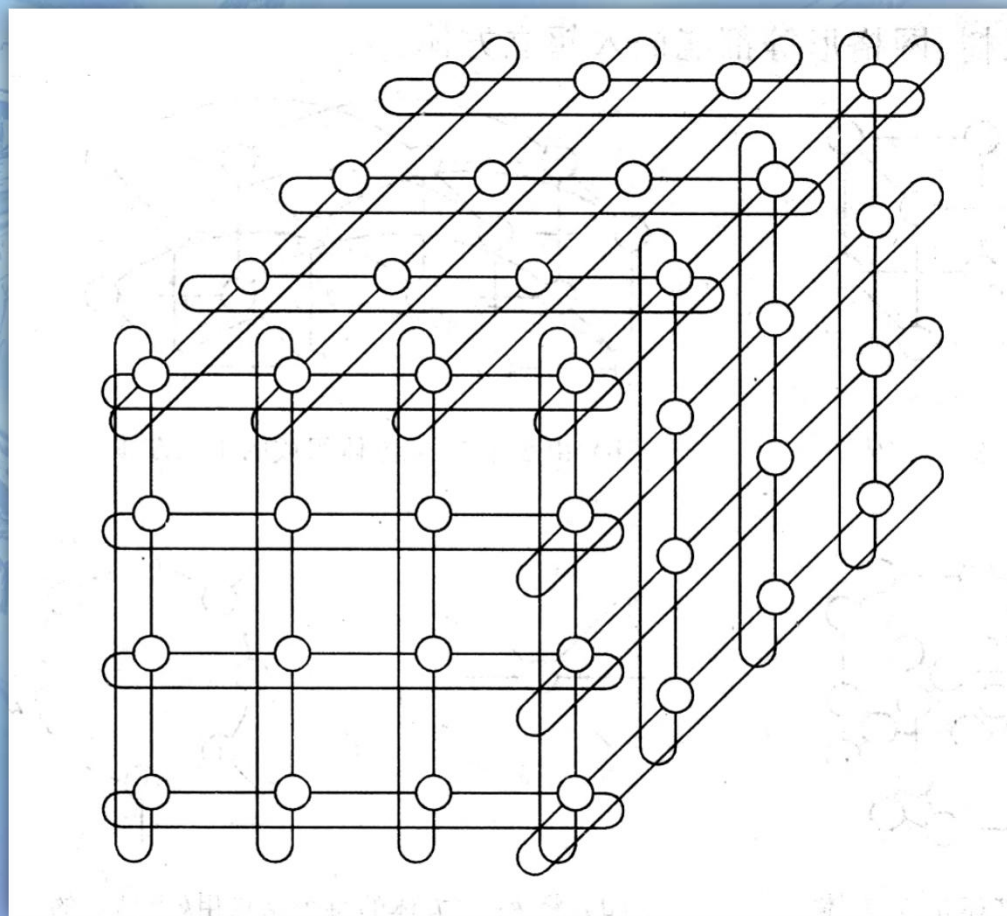
- ◆ 环形、网格、环网形、二元n-立方体（超立方体）和Omega网络都是**k元n-立方体网络**系列的拓扑同构体
- ◆ 在k元n-立方体网络中，**参数n**是立方体的维数，**k**是基数，即每一维上的节点个数

$$N=k^n, \quad (k = \sqrt[n]{N}, \quad n=\log_k N)$$

- ◆ k元n-立方体的节点可以用基数为k的n位地址 **$A = a_1 a_2 \dots a_n$** 来表示，其中 **a_i** 表示该节点在第**i**维上的位置
- ◆ 通常把低维k元n-立方体称为**环网**，而把高维k元n-立方体称为**超立方体**



4元3-立方体网络





总结

网络类型	节点度	网络直径	链路数	等分带宽	对称性	网络规模
线性阵列	2	$N-1$	$N-1$	1	非	N 个节点
环形	2	$\lfloor N/2 \rfloor$	N	2	是	N 个节点
全连接	$N-1$	1	$N(N-1)/2$	$(N/2)^2$	是	N 个节点
二叉树	3	$2(h-1)$	$N-1$	1	非	树高 $h = \lceil \log_2 N \rceil$
星形	$N-1$	2	$N-1$	$\lfloor N/2 \rfloor$	非	N 个节点
2D网格	4	$2(r-1)$	$2N-2r$	r	非	$r \times r$ 网络, $r = \sqrt{N}$
ILLIAC网	4	$r-1$	$2N$	$2r$	非	$r \times r$ 网络, $r = \sqrt{N}$
2D环网	4	$2\lfloor r/2 \rfloor$	$2N$	$2r$	是	$r \times r$ 网络, $r = \sqrt{N}$
超立方体	n	n	$nN/2$	$N/2$	是	N 个节点, $n = \log_2 N$
CCC	3	$2k-1 + \lfloor k/2 \rfloor$	$3N/2$	$N/(2k)$	是	$N = k \times 2^k$ 结点 环长 $k \geq 3$
k 元 n -立方体	$2n$	$n\lfloor k/2 \rfloor$	nN	$2k^{n-1}$	是	$N = k^n$ 个结点

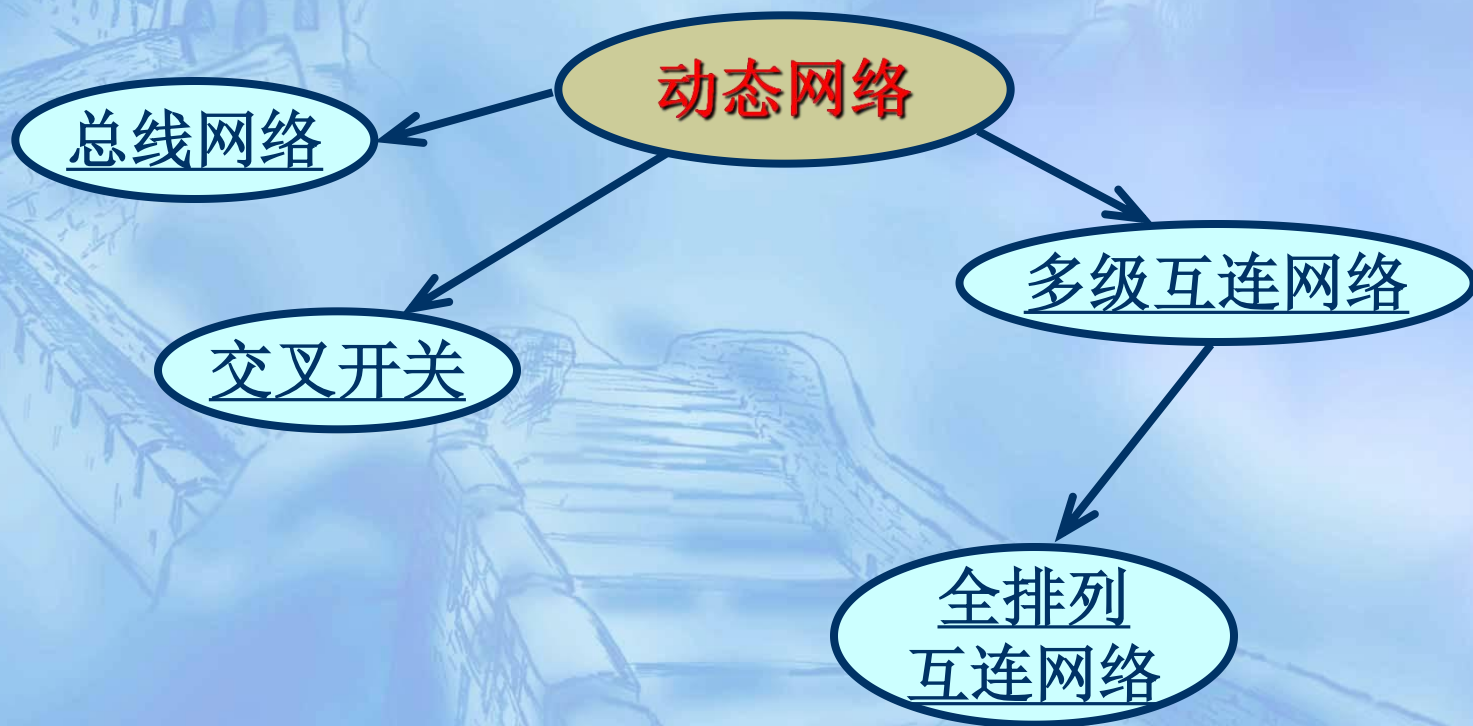


动态互连网络

动态互连网络中的连接通路是可变的，
节点之间的连接可以重新配置。一般由开关
和连线组成，通过控制开关来建立不同的连
接。



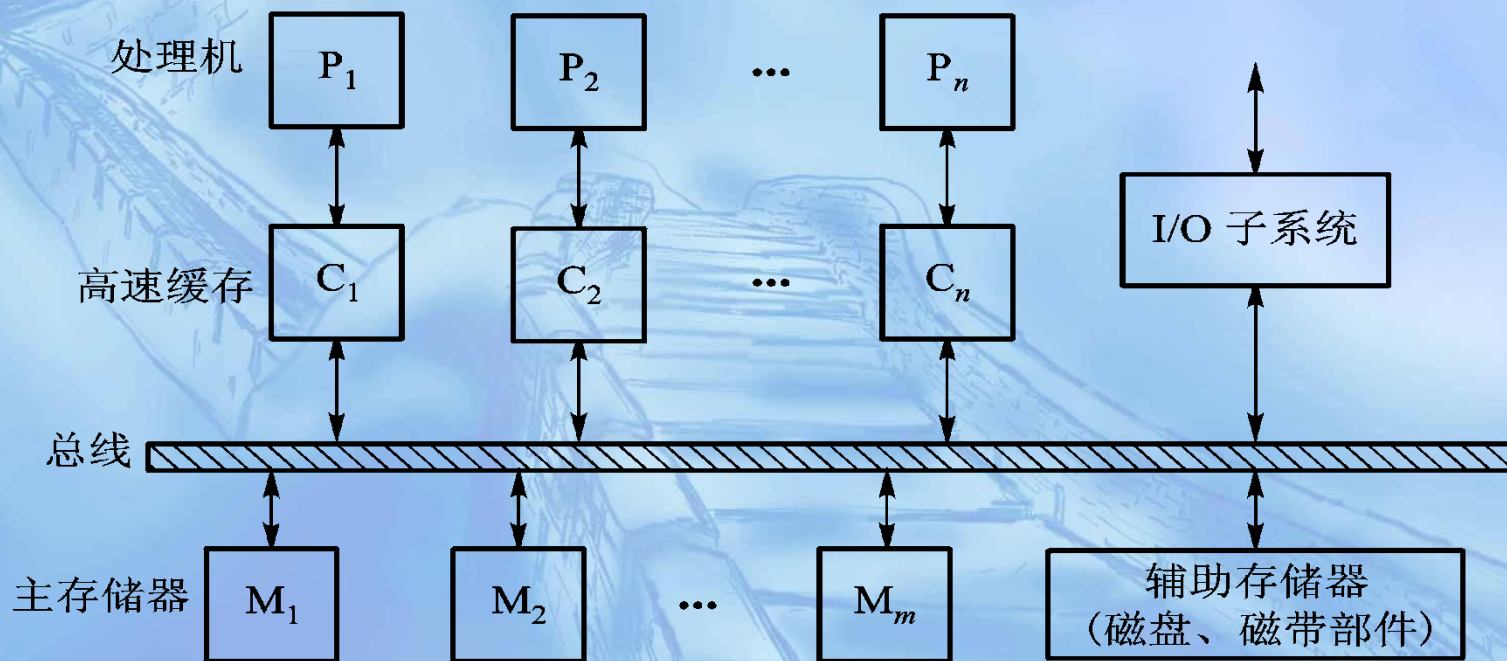
分类





总线网络

多个处理机、存储器模块和外围设备通过接口与公用总线相连，多个节点将争用总线，**必需裁决或分时使用**。但连接可以任意、动态地变化。





问题及解决

问题：公用总线上的节点数目增多会导致系统效率急剧下降。

解决：

- ◆ 采用优质高频同轴电缆来提高总线的传输速率；
- ◆ 采用多总线方式来减少访问总线的冲突概率。

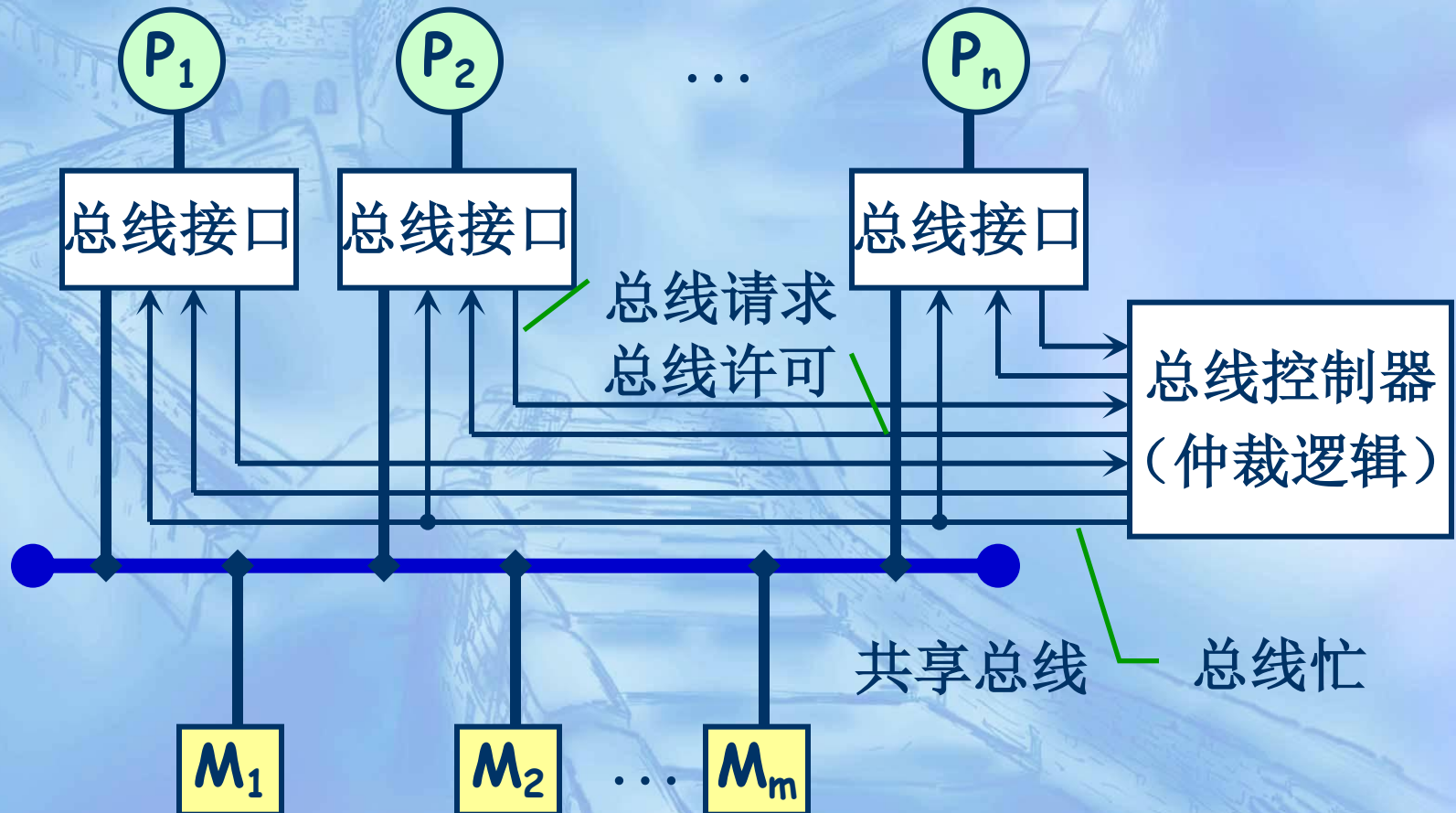
问题：存在访问冲突。

解决：

- ◆ **静态优先级算法：**为连接到总线的每个部件分配一个固定的优先级。
- ◆ **固定时间片算法：**将总线按时间分成固定大小的时间片，轮流提供给部件使用。
- ◆ **动态优先级算法：**让总线上各部件优先级根据情况和一定规则动态地改变。例如：**LRU**。
- ◆ **先来先服务算法：**按接受到访问总线请求的先后顺序来响应。



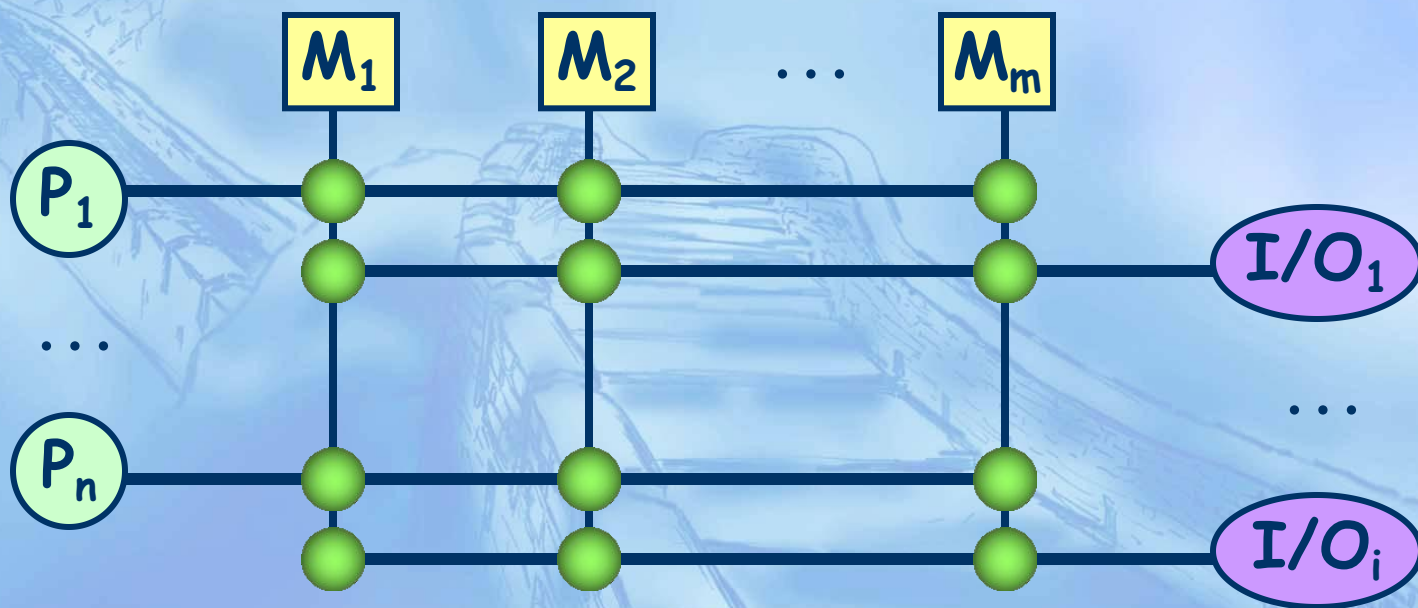
举 例





交叉开关网络

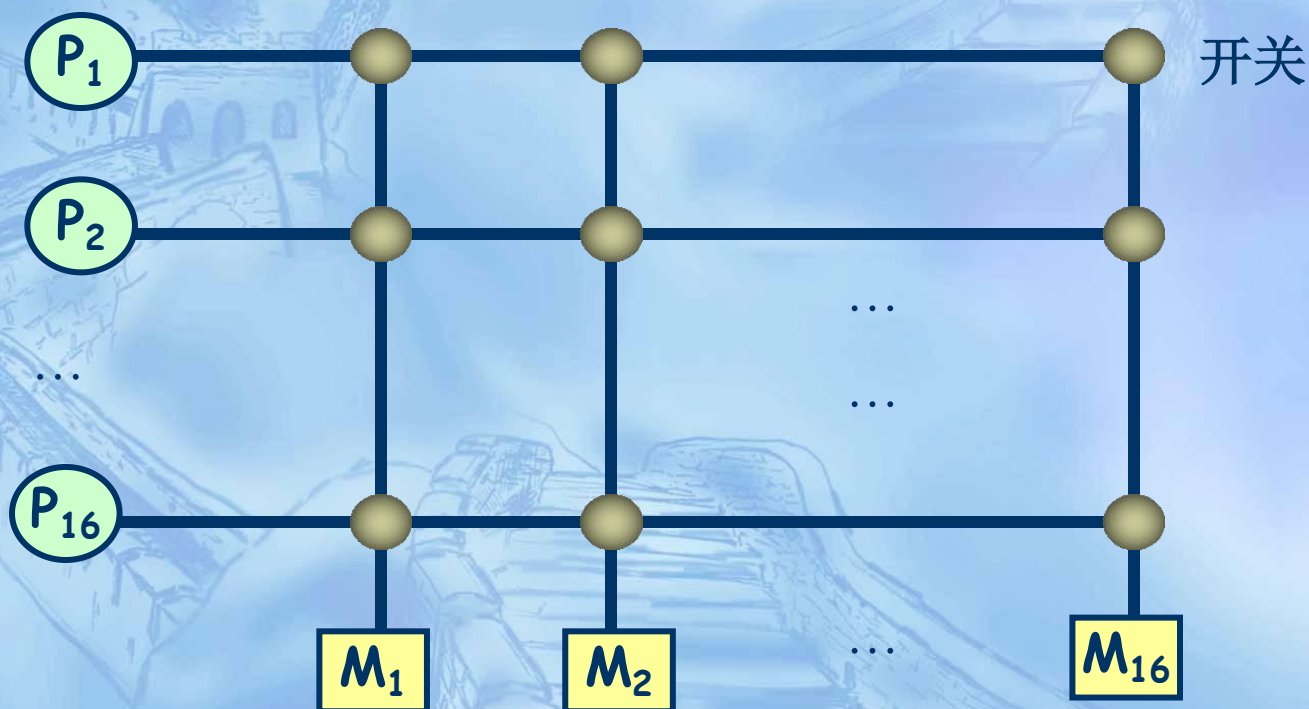
一组纵横开关阵列构成多总线（总线数为 $m+n+i$ ，且 $m \geq n+i$ ；可同时通信的总线数为 m ），交叉点开关能在对偶（源和目的）之间形成动态连接。





举 例

— C.mmp多处理机



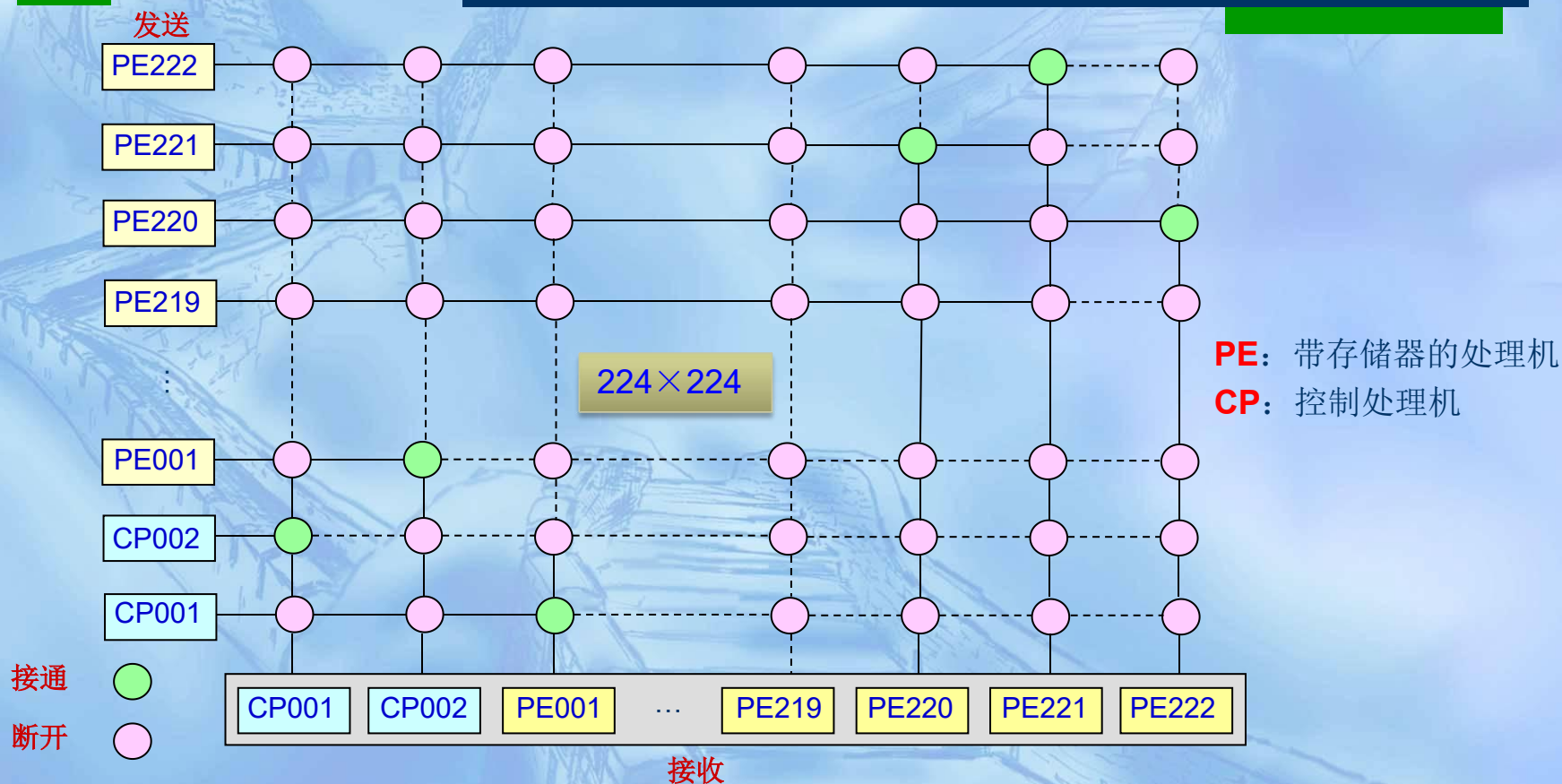
处理器-存储器交叉开关网络

每列只能接通一个交叉点开关，同一行中可以接通几个交叉点开关



举 例

一向量并行处理机VPP500



处理机间的交叉开关网络，每行和每列只能接通一个交叉点开关（一对一连接）



问题及解决

问题:

交叉开关网络很复杂，代价大（因此一般只有 $n \leq 16$ 的小型交叉开关网络用在商品化的计算机中）。因为开关阵列的每个交叉点上均有一套开关，每套开关内部含有多选一的仲裁模块和多路转换器模块。

解决:

通过用多个较小规模的交叉开关网络“串联”和“并联”来构成**多级交叉开关网络**，以取代单级的大规模的交叉开关。



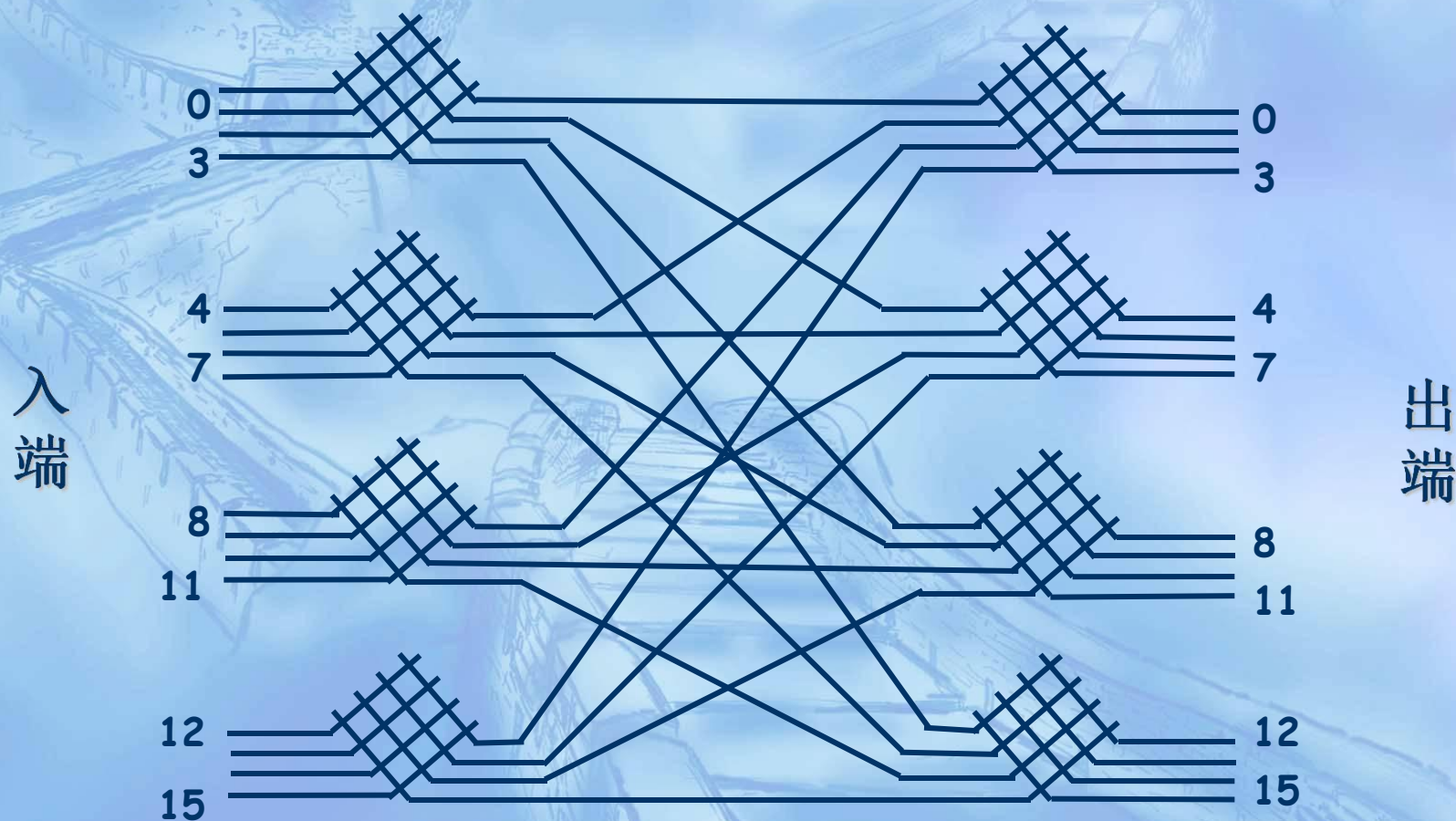
Delta网

用 $a \times b$ 的交叉开关模块构成 $a^n \times b^n$ 的交叉开关网络，其中指数 n 为互连网络的级数。

例如：用 4×4 的交叉开关模块构成 $4^2 \times 4^2$ 的交叉开关网络，其中指数 2 为互连网络的级数。



图 示





多级互连网络

◆ 基本思想

◆ 关键技术

- 开关模块
- 控制方式
- 级间连接模式

◆ 常见多级网络

- 多级立方体网络
 - ✓ STARAN网络
 - ✓ 间接二进制n方体网络
- Omega (Ω) 网络
- 基准网络

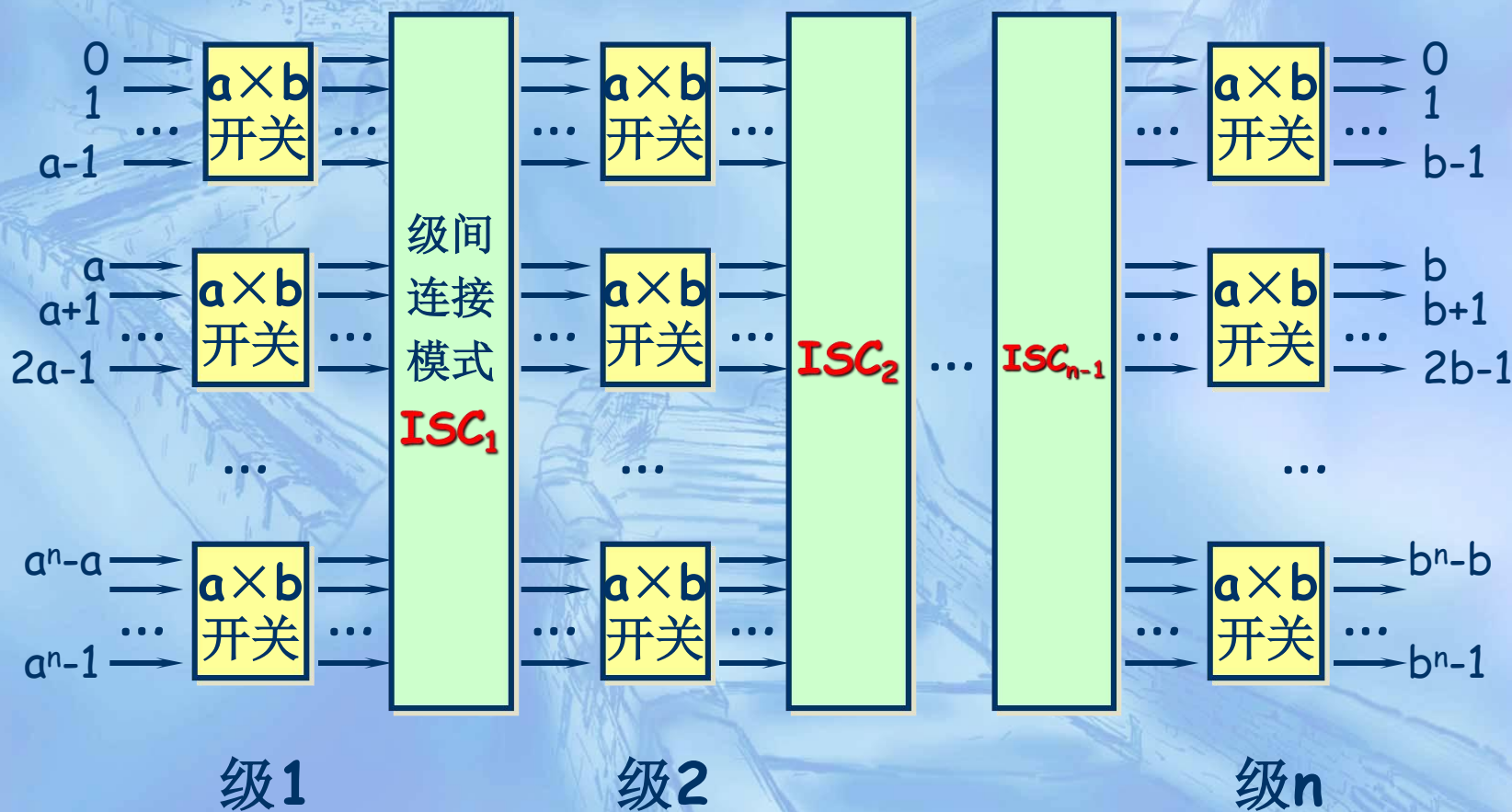


基本思想

多级互连网络用若干个较小规模的**开关模块**组成**开关级**，开关级之间有固定连接的**级间连接**，通过**控制信号**改变开关模块的输入端与输出端之间的连接状态，从而可改变网络输入端与输出端之间的连接。



通用多级网络结构





开关模块

一个 $a \times b$ 开关模块有 a 个输入和 b 个输出，最常用的是**二元开关**（ 2×2 ）。每个输入可与一个或多个输出相连，但是在输出端必须避免发生冲突。即：一对一和一对多映射是容许的，但不容许有多对一映射。

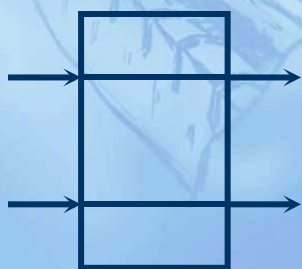
只容许一对一映射时称为**置换连接**，称这种开关为 **$n \times n$ 交叉开关**。

模块大小	合法状态	置换连接
2×2	4	2
4×4	256	24
8×8	16777216	40320
$n \times n$	n^n	$n!$

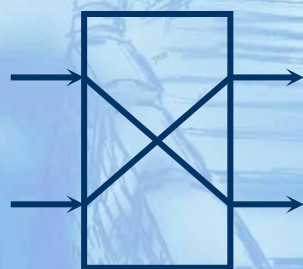


二元开关

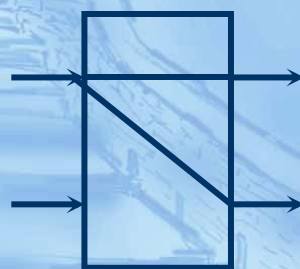
具有直通和交换两种功能的二元开关称为**二功能开关**，或**交换开关**，用一位控制信号控制。具有所有四种功能的二元开关称为**四功能开关**，用两位控制信号控制。



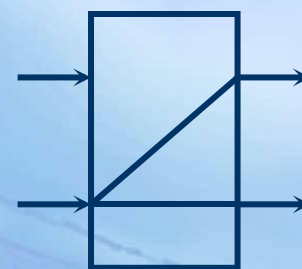
直通



交换



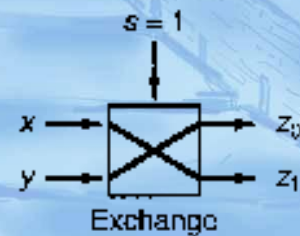
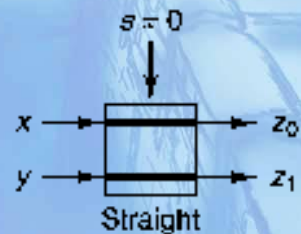
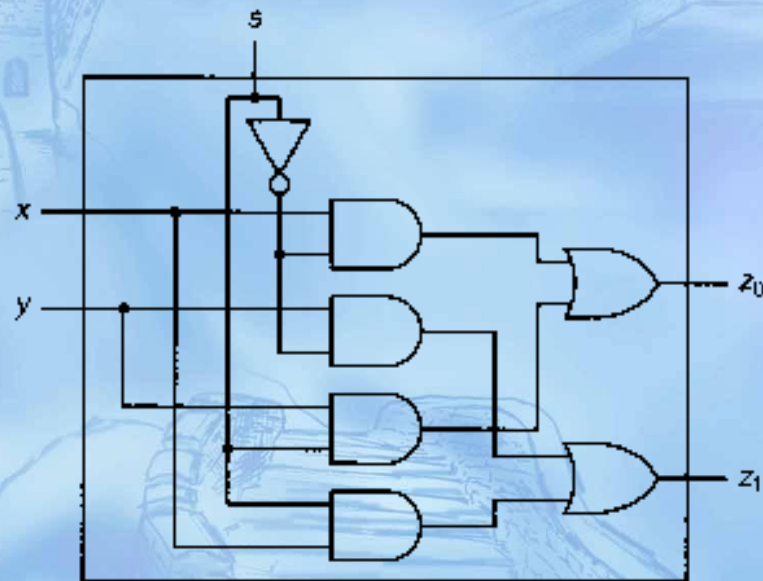
上播



下播



二功能开关的实现





控制方式

在多级互连网络中有多级开关模块，每一级又有多个开关模块。通常有三种控制方式：

- ◆ **级控制**

同一级开关模块使用同一个控制信号控制，只能同时处于同一种状态。

- ◆ **单元级控制**

每个开关模块都有一个独立的控制信号，可各自处于不同的状态。

- ◆ **部分级控制**

例如，第 i 级使用 $i+1$ 个控制信号控制 ($0 \leq i \leq n-1$, n 为级数)。



级间连接模式

级间连接模式是指前一级开关模块的输出端与后一级开关模块的输入端之间的连接模式，也称为**拓扑结构**。通常，采用前面介绍过的互连函数（例如：均匀洗牌、蝶式等）实现拓扑结构。

实际上，从节点的输入到第一级开关模块的输入，以及从最后一级开关模块的输出到节点的输出也可以采用拓扑结构连接。



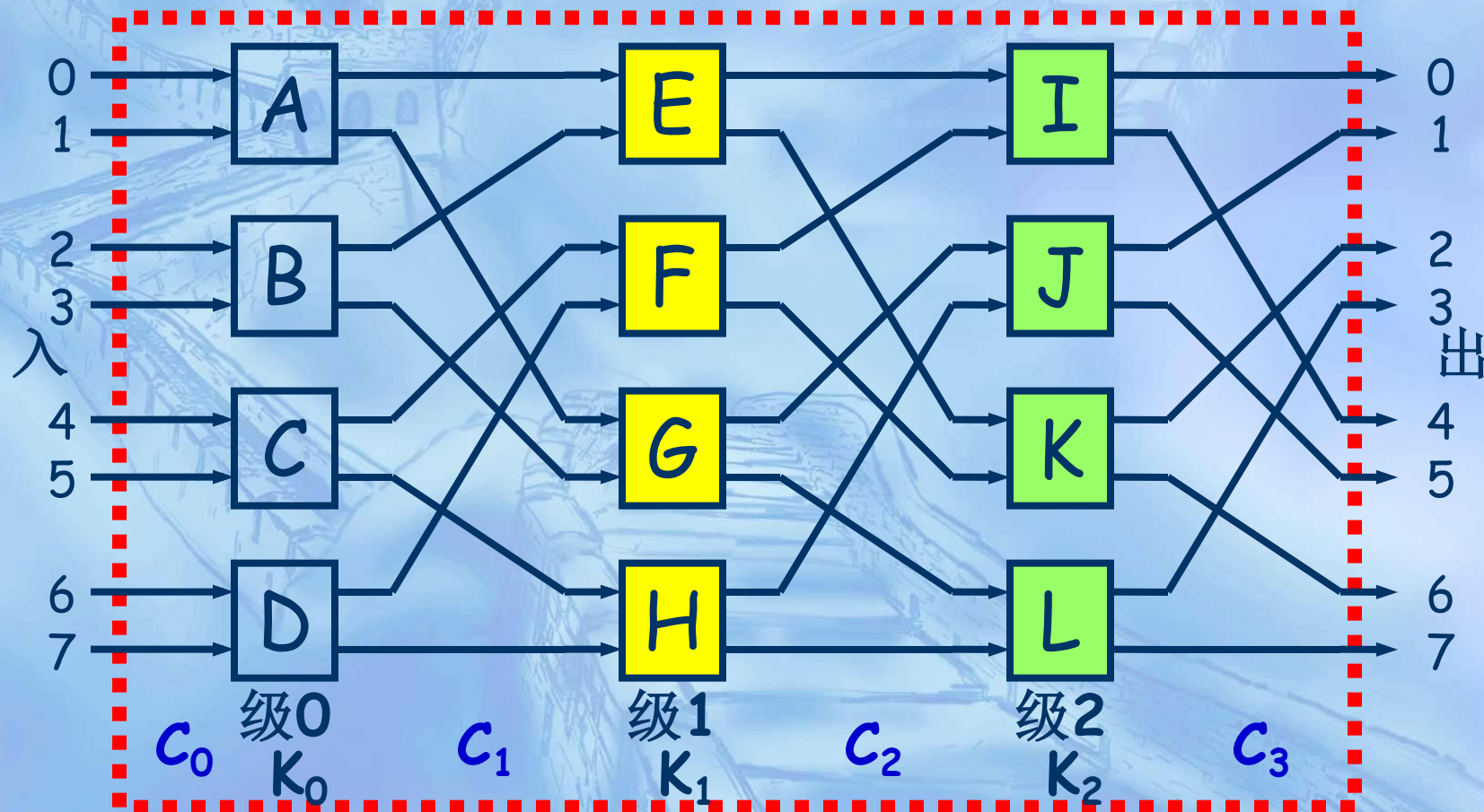
STARAN网络

采用 2×2 的二功能开关（交换开关），对于 $N \times N$ 网络，有 $n = \log_2 N$ 个开关级，每级有 $N/2$ 个交换开关； n 个开关级从输入端到输出端依次为 K_0 、 K_1 、... K_{n-1} ， $n+1$ 个级间连接依次为 C_0 、 C_1 、... C_n ，其中 C_0 为恒等置换， $C_1 \sim C_n$ 都是逆洗牌置换；开关有两种控制方式：级控制方式（**交换网络**）和部分级控制方式（**移数网络**）。



网络结构

(N=8)



		级控制信号 ($k_2k_1k_0$): $k=1$ 为交换, $k=0$ 为直通							
		000	001	010	011	100	101	110	111
入 端	0	0	1	2	3	4	5	6	7
	1	1	0	3	2	5	4	7	6
	2	2	3	0	1	6	7	4	5
	3	3	2	1	0	7	6	5	4
	4	4	5	6	7	0	1	2	3
	5	5	4	7	6	1	0	3	2
	6	6	7	4	5	2	3	0	1
	7	7	6	5	4	3	2	1	0
交换功能 (一组元素首尾对称地进行交换)		i	Cube ⁰	Cube ¹	Cube ⁰ + Cube ¹	Cube ²	Cube ⁰ + Cube ²	Cube ¹ + Cube ²	Cube ⁰ + Cube ¹ + Cube ²
		恒等	4组2元	4组2元 + 2组4元	2组4元	2组4元 + 1组8元	4组2元 + 2组4元 + 1组8元	4组2元 + 1组8元	1组8元

STARAN交换网络

2级	K,L	0	0	1	0	0	0	0
	J	0	1	1	0	0	0	0
	I	1	1	1	0	0	0	0
1级	G,H	0	1	0	0	1	0	0
	E,F	1	1	0	1	1	0	0
0级	A,B,C,D	1	0	0	1	0	1	0
入 端	0	1	2	4	1	2	1	0
	1	2	3	5	2	3	0	1
	2	3	4	6	3	0	3	2
	3	4	5	7	0	1	2	3
	4	5	6	0	5	6	5	4
	5	6	7	1	6	7	4	5
	6	7	0	2	7	4	7	6
	7	0	1	3	4	5	6	7
功 能		移1 mod 8	移2 mod 8	移4 mod 8	移1 mod 4	移2 mod 4	移1 mod 2	不移 衡等

STARAN移数网络



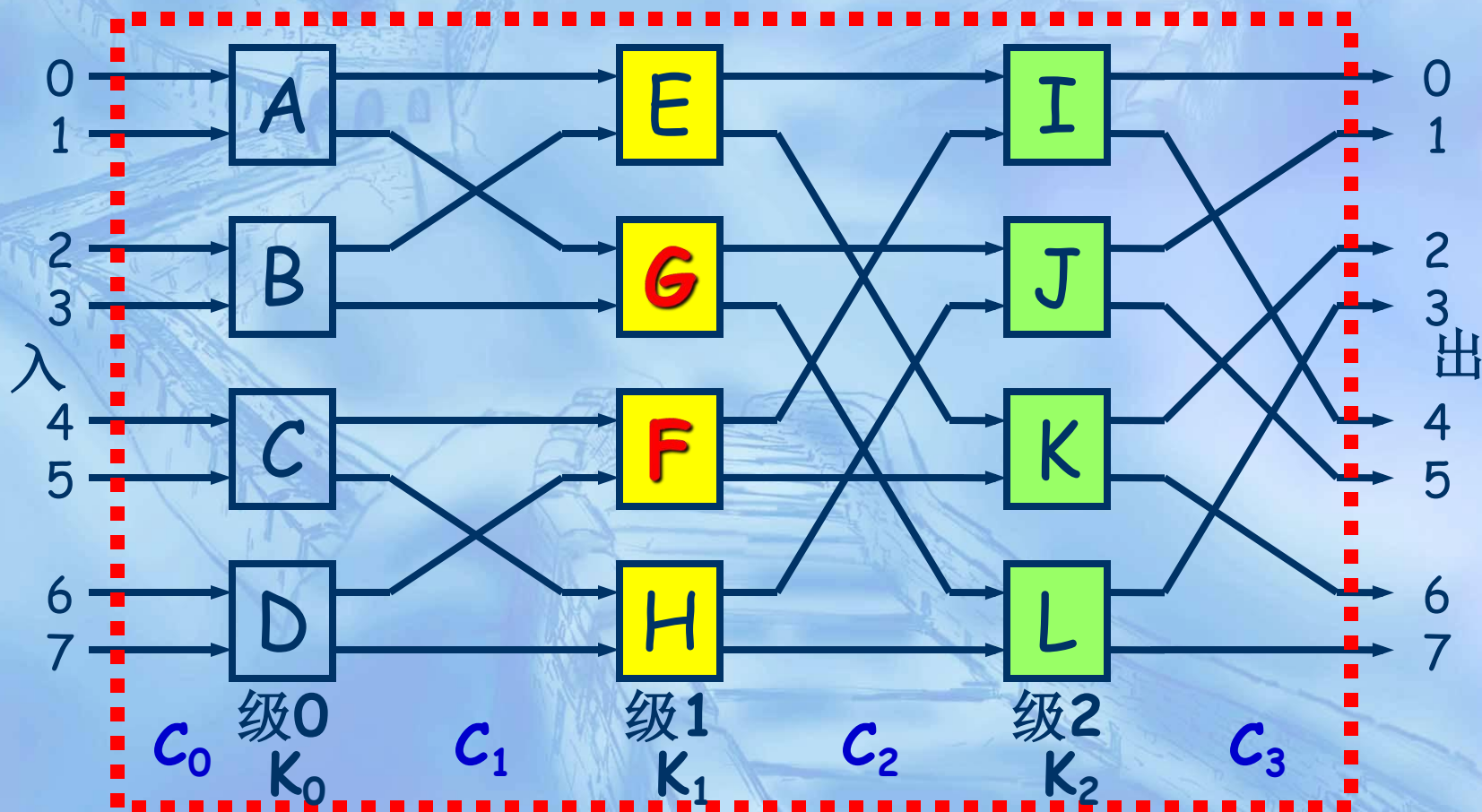
间接二进制n方体网络

采用 2×2 的二功能开关（交换开关），对于 $N \times N$ 网络，有 $n = \log_2 N$ 个开关级，每级有 $N/2$ 个交换开关； n 个开关级从输入端到输出端依次为 K_0 、 K_1 、... K_{n-1} ， $n+1$ 个级间连接依次为 C_0 、 C_1 、... C_n ，其中 C_0 为恒等置换， $C_1 \sim C_{n-1}$ 都是子蝶式置换，且 $C_i = \beta_{(i+1)}$ ， C_n 为逆洗牌置换；开关采用单元控制方式。



网络结构

($N=8$)





提示

STARAN网络和**间接二进制n方体网络**都是**多级立方体网络**，因为当第*i*级（ $0 \leq i \leq n-1$ ）交换开关处于交换状态时实现的是**Cube_i**函数。

两个网络很相似（**F、G交换位置**），唯一不同之处在于：**交换开关的控制方式不同**，前者采用级控制和部分级控制方式，后者采用单元控制方式。



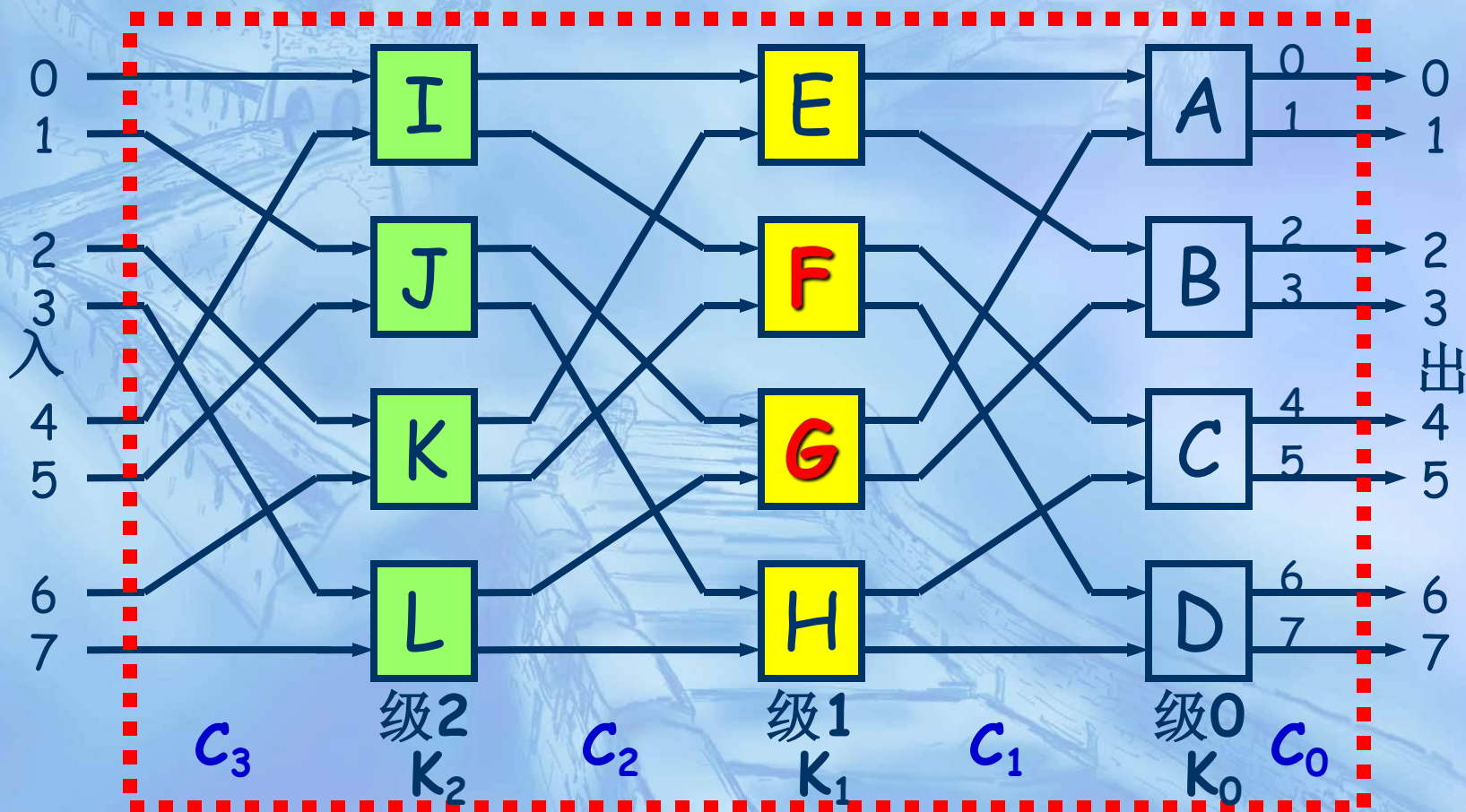
Omega网络

采用 2×2 的四功能开关，对于 $N \times N$ 网络，有 $n = \log_2 N$ 个开关级，每级有 $N/2$ 个开关； n 个开关级从输入端到输出端依次为 K_{n-1} 、 $\dots$ K_1 、 K_0 ， $n+1$ 个级间连接依次为 C_n 、 $\dots$ C_1 、 C_0 ，其中 C_0 为恒等置换， $C_1 \sim C_n$ 都为均匀洗牌置换；开关采用单元控制方式。本网络也称为：**多级洗牌置换网络或多级混洗网络。**



网络结构

(N=8)





提示

Omega网络 (Ω 网络) 可看作是多级立方体网络的逆网络。

- ◆ **级控制且开关为二功能开关**

Ω 网络是STARAN交换网络的逆网络。

- ◆ **部分级控制且开关为二功能**

Ω 网络是STARAN移数网络的逆网络。

- ◆ **单元控制且开关为二功能**

Ω 网络是间接二进制n方体网络的逆网络。



基准网络

采用 2×2 的二功能开关（交换开关），对于 $N \times N$ 网络，有 $n = \log_2 N$ 个开关级，每级有 $N/2$ 个交换开关； n 个开关级从输入端到输出端依次为 K_0 、 K_1 、... K_{n-1} ， $n+1$ 个级间连接依次为 C_0 、 C_1 、... C_n ，其中 C_0 和 C_n 为恒等置换， C_1 为逆洗牌置换 δ^{-1} ， $C_2 \sim C_{n-1}$ 都是逆子洗牌置换，且 $C_i = \delta^{-1}_{(n-i+1)}$ ， $2 \leq i \leq n-1$ ；开关采用单元控制方式。



全排列互连网络

在同时实现两对或多对入端和出端之间的连接时，有可能因争用数据传送通路而发生冲突的网络称为**阻塞型网络**。反之，将能够实现所有可能的入、出端间的连接而不发生冲突的互连网络称为**非阻塞型网络或全排列网络**。



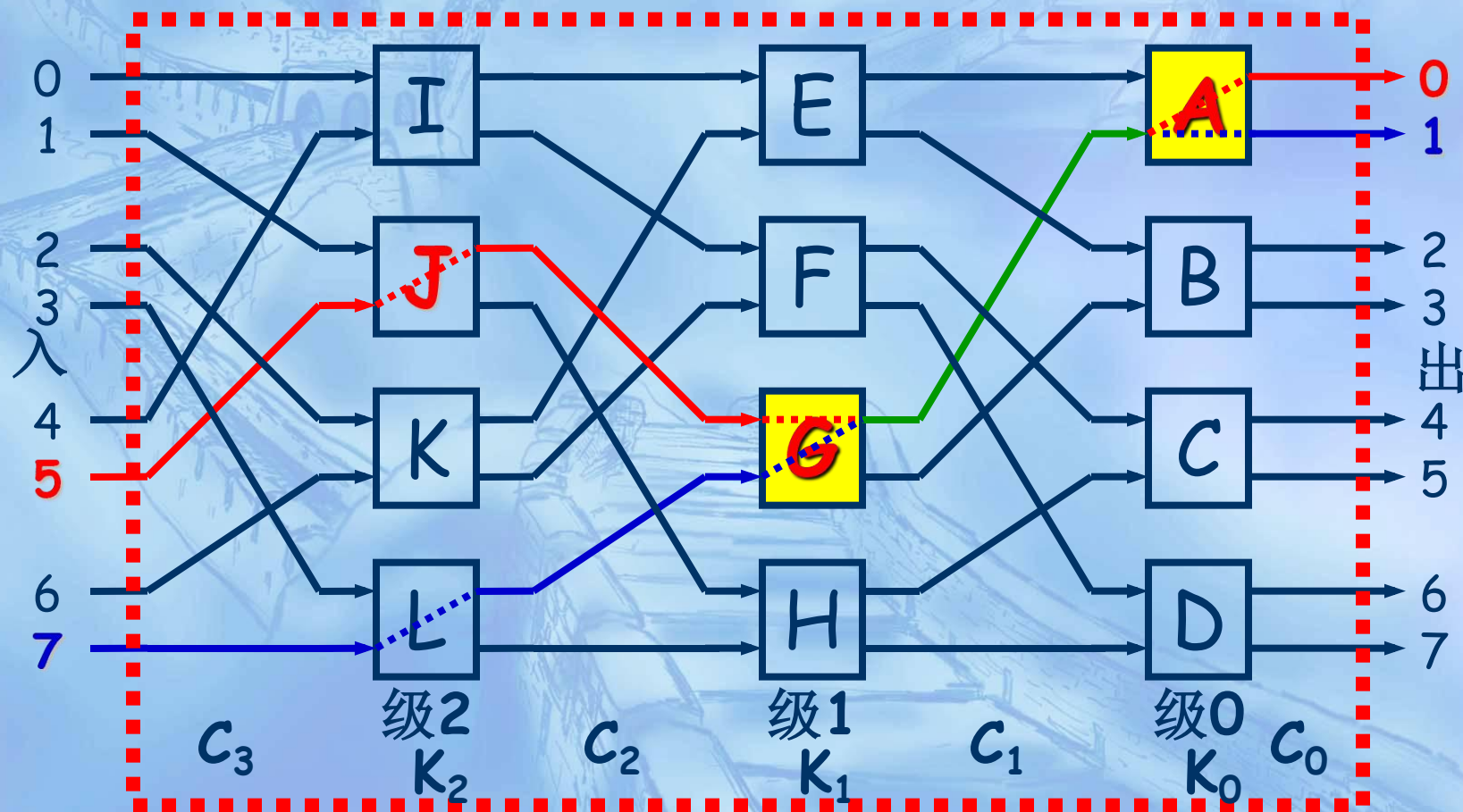
阻塞型网络举例

例如：在**Omega**网络中，如果同时进行 $5 \rightarrow 0$ （**JGA=101**）和 $7 \rightarrow 1$ （**LGA=110**）连接时，产生冲突，不能同时进行（见后图）；则**Omega**网络就是阻塞型网络。

其实前面介绍的各种多级网络都是阻塞型网络。



图 示





原因分析

多级网络的排列数满足不了要求。以间接二进制n方体网络为例：该互连网络共有 $n = \log_2 N$ 级，每级 $N/2$ 个交换开关，共有 $\log_2 N * N/2$ 个开关，每个开关只有直连和交换两个功能，故组合数为 $2^{\log_2 N * N/2}$ ，即 $N^{N/2}$ ，小于实现N个处理单元所需N! 组合数。



全排列网络的实现

◆ 多次经过同一多级网络

将这种只要经过重新排列已有入出端对的连接，就可以完成所有可能的入出端间的连接而不发生冲突的互连网络称为**可重排列网络**。

◆ 多个多级网络串联使用

例如：将多级立方体网络和它的逆网络连在一起，省去中间重复的一级后，构成的全排列网络称为**Benes网络**。



消息传递机制

- ◆ 消息寻径方式
- ◆ 死锁和虚拟通道
- ◆ 流控制策略



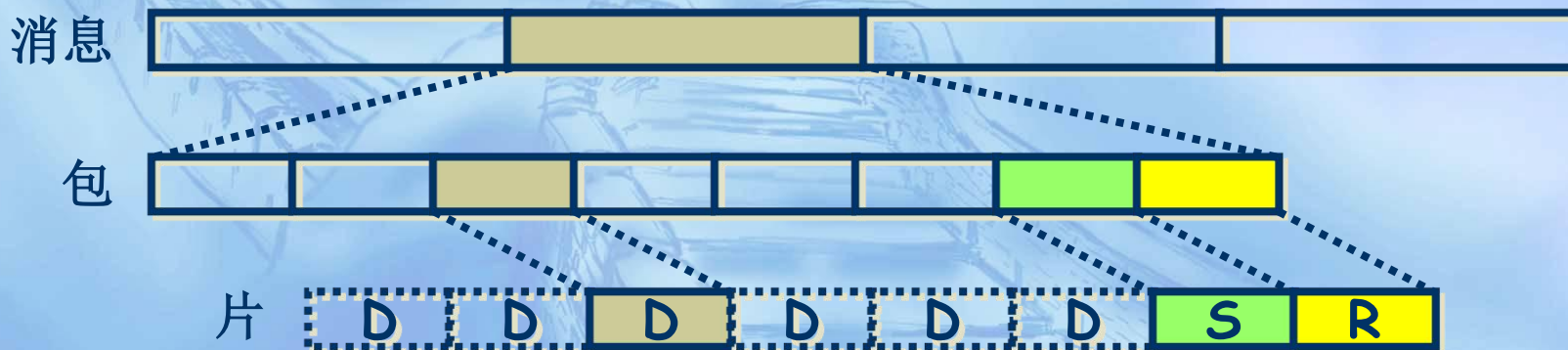
消息寻径方式

- ◆ 消息格式
- ◆ 寻径方式
 - 线路交换寻径
 - 包交换寻径
 - ✓ 存储转发寻径
 - ✓ 虚拟直通寻径
 - ✓ 虫蚀寻径



消息格式

- ◆ **消息 (Message)**：节点间通信的逻辑单位，长度任意。
- ◆ **包 (Packet)**：包含寻径所需目的地址的基本单位，内含路由信息及序号，长度固定，典型值为64~512bit。
- ◆ **片 (Slice)**：更小的单位，由包分解而得，长度固定（例如：8bit），可分为路由片、序号片、数据片。



D: 数据; S: 序号; R: 路由;



线路交换寻径 (circuit switch)

◆ 方法

先建立一条从源节点到目的节点的物理通路，然后再传递消息。

◆ 优点

传输带宽较大，平均传输时延较小，使用缓冲区少。适合于具有动态和突发性的大规模并行处理数据的传送。

◆ 缺点

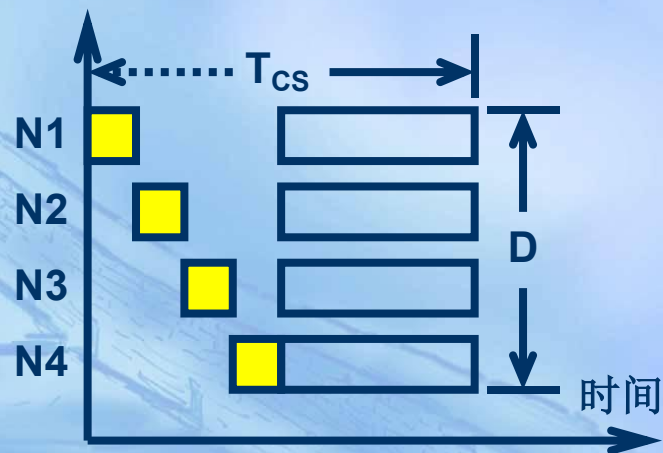
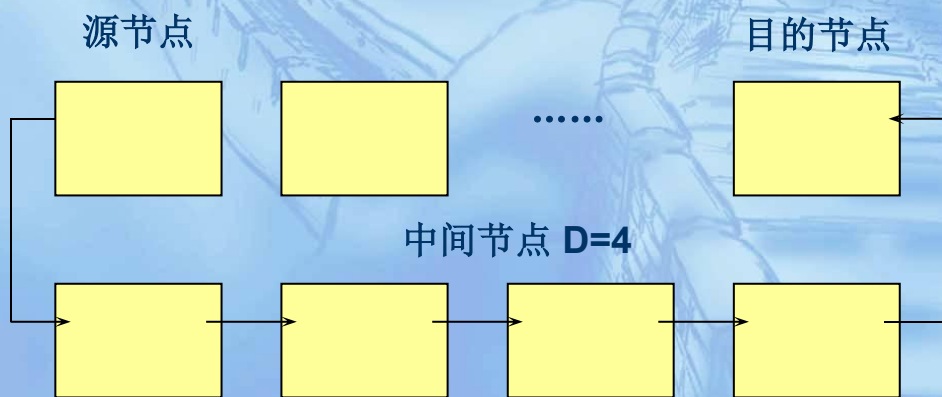
建立源节点到目的节点的物理通路开销很大，占用物理通路的时间长。



传输时延

$$T_{CS} = (L_t / B) \times (D + 1) + L / B$$

其中： L_t 为建立路径所需小信息包的长度； L 为信息包的长度； D 为经过的中间节点数； B 为带宽。





存储转发寻径 (store and forward)

◆ 方法

信息流的基本单位是包，每个节点有一个包缓冲区，包从源节点经过中间节点（存储 - 转发）到达目的节点。

◆ 优点

占用物理通路的时间比较短。

◆ 缺点

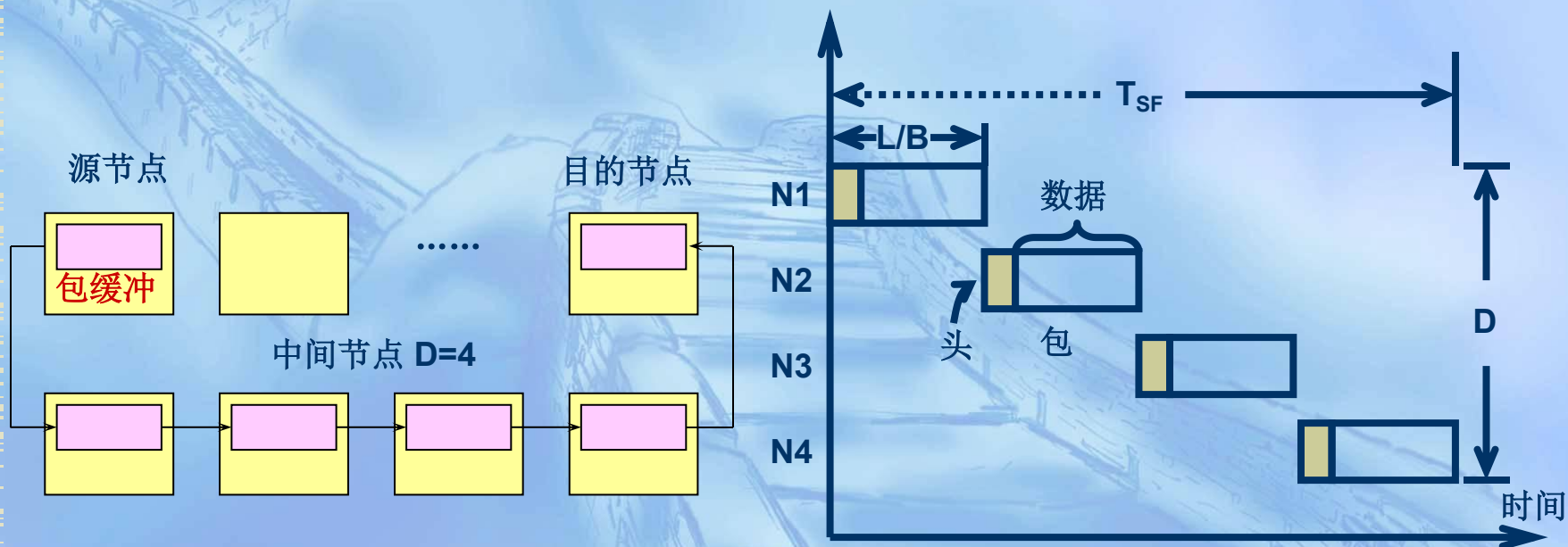
包缓冲区大（不利于VLSI实现），时延大（与节点距离成正比）。



传输时延

$$T_{SF} = (L / B) \times (D + 1)$$

其中：**L**为信息包的长度；**D**为经过的中间节点数；**B**为带宽。





虚拟直通寻径 (virtual cut through)

◆ 方法

当接收到用作寻径的包头部时，即开始路由选择，这样可以降低时延。（是对**存储转发寻径**的改进）

◆ 优点

通信时延与节点数无关。最佳情况下的通信时延同“线路交换寻径”（整条链路都空闲时）。

◆ 缺点

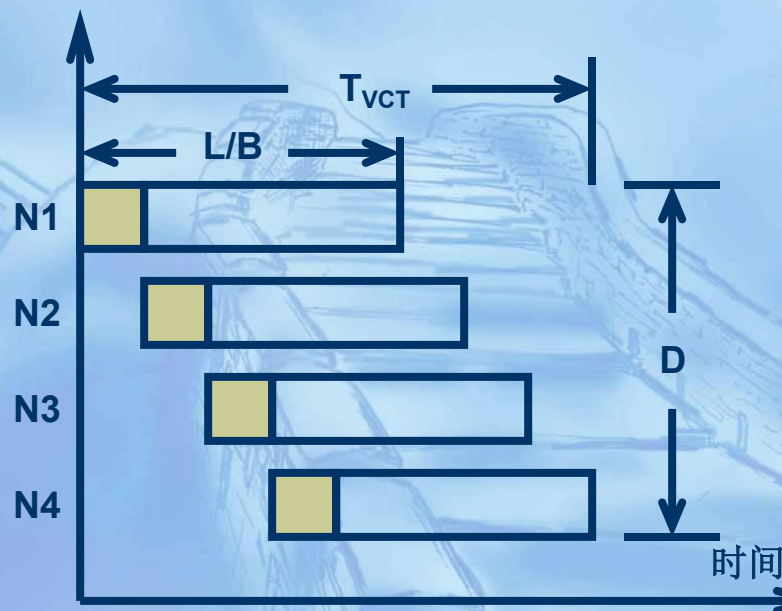
出现寻径阻塞时只能将整个包存储在寻径节点中，故节点皆需足够大的缓冲区来存储最大包。最坏情况下的通信时延同“存储转发寻径”，此时经过的每个节点都发生阻塞，都需缓冲。



传输时延

$$T_{VCT} = (L_h / B) \times D + L/B$$

其中： L_h 为包寻径头部的长度； L 为信息包的长度； D 为经过的中间节点数； B 为带宽。一般有： $L \gg L_h \times D$ ，通信时延可以近似为： $T_{VCT} = L/B$ ，与节点数无关。





虫蚀寻径 (wormhole)

将包分成更小的片，在每个节点的寻径器中设置**片缓冲区**。用头片直接开辟一条从输入节点到输出节点的路径，每个包中的片以**流水方式**在网络中向前“蠕动”（是对**虚拟直通寻径**的改进）。

当包的头片到达一个节点**A**的寻径器后，寻径器根据头片的寻径消息立即做出路由选择。如果所选择的通道或节点的片缓冲区不可用时，头片必须在该节点的片缓冲区中等待，其它数据片也在原来的节点上等待。

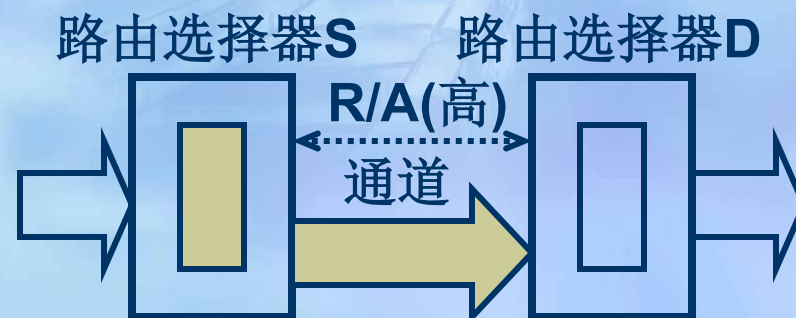


握手协议

R/A(就绪/请求)线



(a) D已就绪可以接收一片



(b) S已就绪准备发送片i



(c) D接收片i



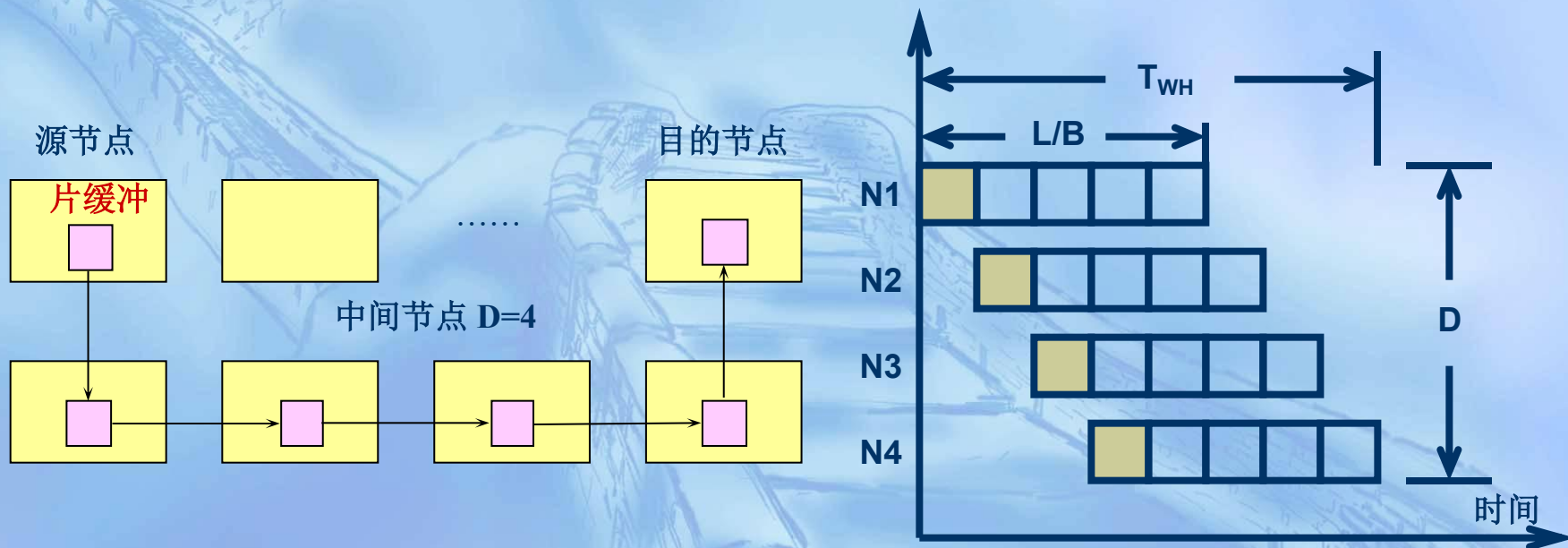
(d)片i从D中移走, 片i+1到达S的片缓冲区



传输时延

$$T_{WH} = (L_f / B) \times D + L/B$$

其中： L_f 为片的长度； L 为信息包的长度； D 为经过的节点数； B 为带宽。一般有： $L \gg L_f \times D$ ，通信时延可以近似为： $T_{WH} = L/B$ ，与节点数无关。





特点

虫蚀寻径的优点:

- 每个节点的缓冲区较小（易于VLSI实现）；
- 较低的网络传输时延：存储转发的消息包整个从一个节点“跳到”另一个节点，通道的使用是串行的，它的传输时延基本上正比于消息在网络中传输的距离；而虫蚀寻径所有的片以流水方式向前传输，采用了时间并行性，它的传输时延正比于消息包的长度，传输距离对它的影响很小；
- 通道共享性好，利用率高；
- 易于实现选播和广播通信方式；

虫蚀寻径的缺点:

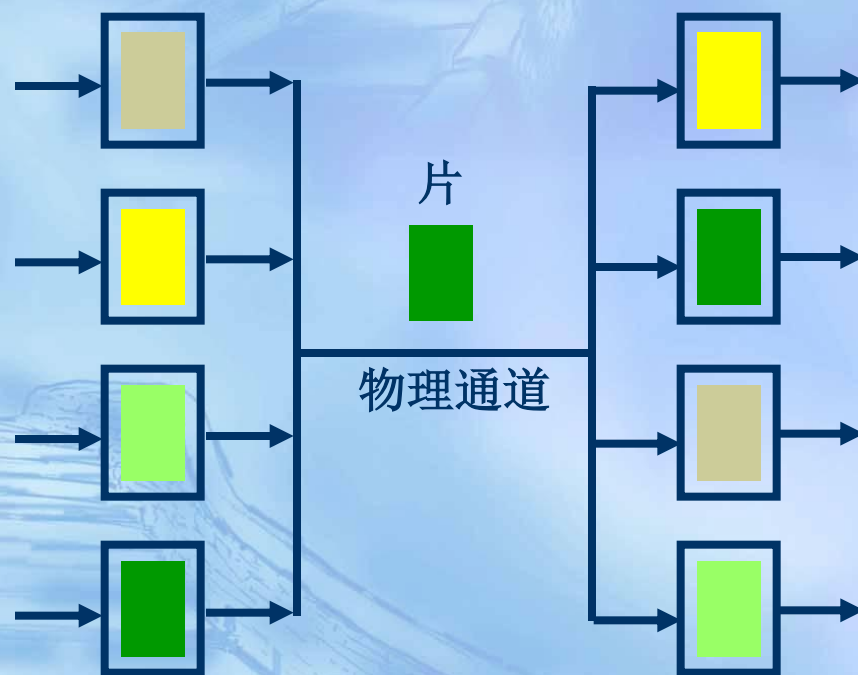
当包的一个片被阻塞时，整个包都被阻塞在所有节点，占用了节点资源。



虚拟通道

虚拟通道是两个节点间的逻辑链路，由源节点的片缓冲区、节点间的物理通道及接收节点的片缓冲区组成。这可增加利用率。

源节点中的片缓冲区 目的节点中的片缓冲区



四条虚拟通道以片传输为基础
分时共用一条物理通道



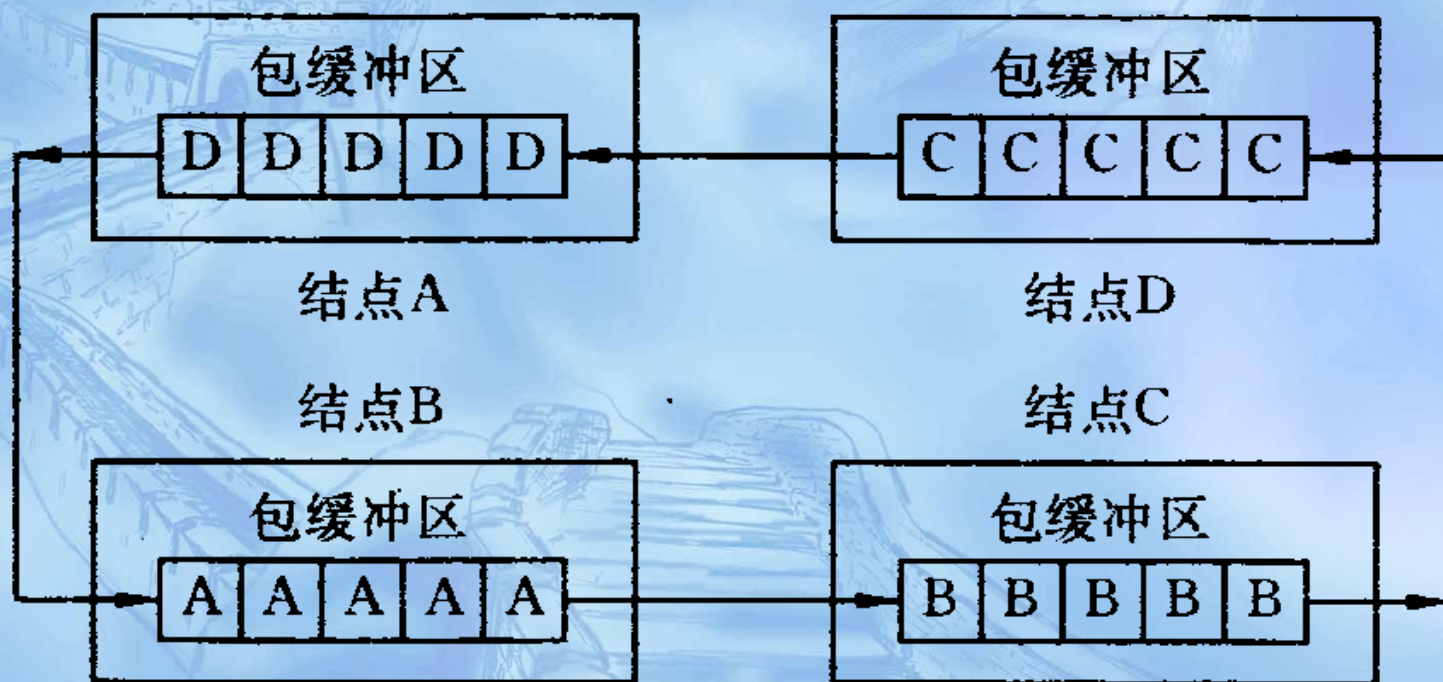
死锁的产生

缓冲区或通道上的循环会引起死锁。下面通过两个例子进行说明：

- ◆ 采用存储转发寻径时
- ◆ 采用虫蚀寻径时



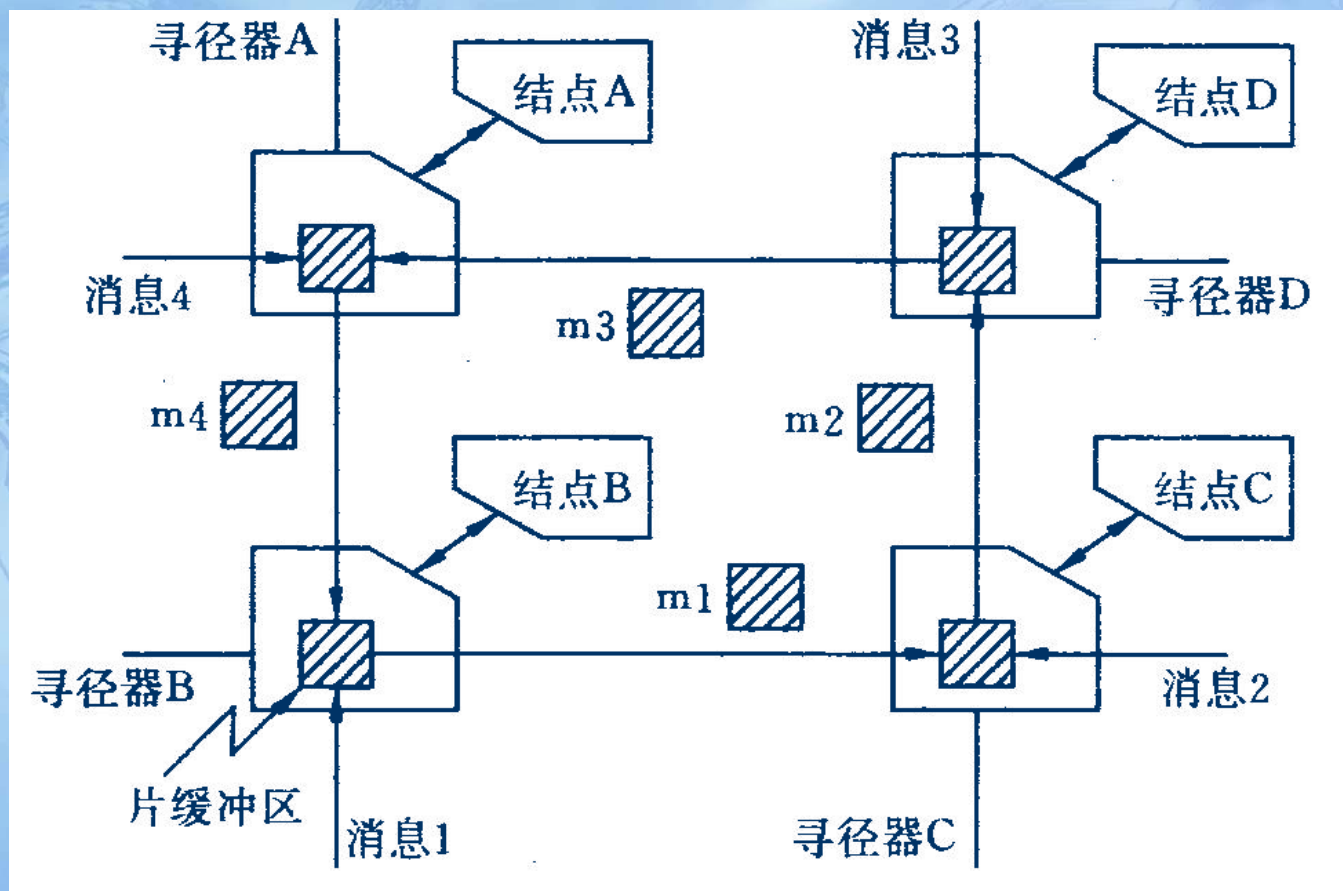
采用存储转发寻径时



四个消息包占用了四个节点的缓冲区，导致循环等待



采用虫蚀寻径时

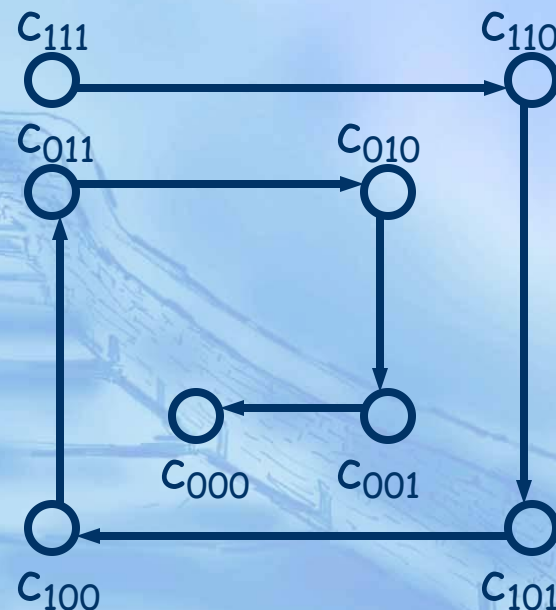
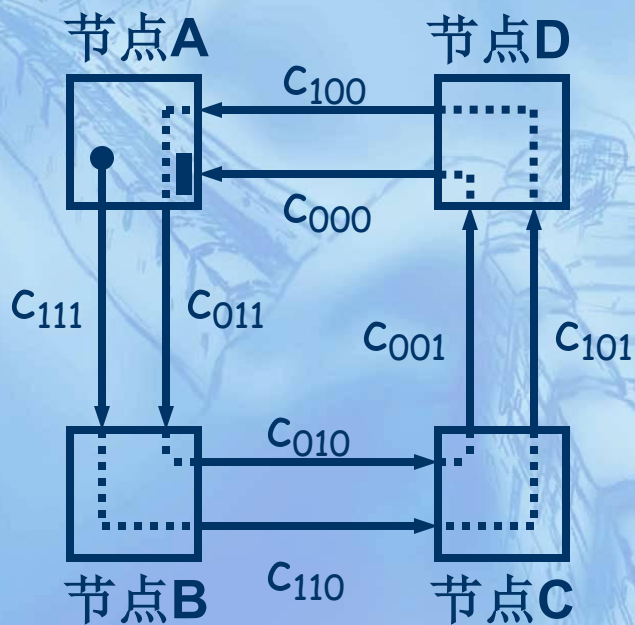


四个消息沿四条通道同时传送也会产生通道死锁



死锁的避免

利用虚拟通道打破循环，可以避免死锁。每条物理通道包含两条虚拟通道（左图），利用虚拟通道修正后的通道图见右图，永远不成循环。



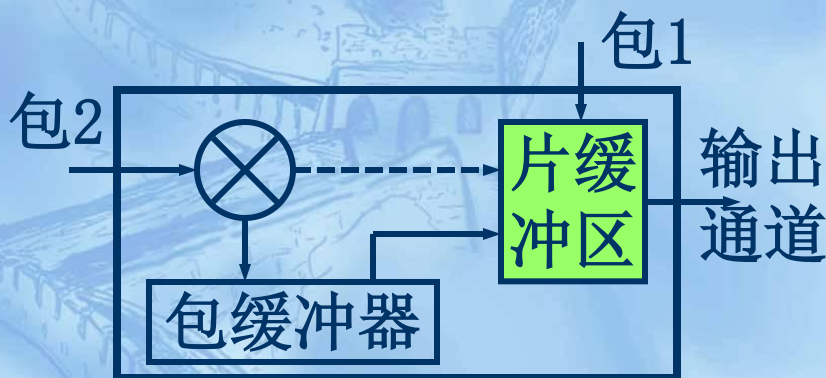


流控制策略

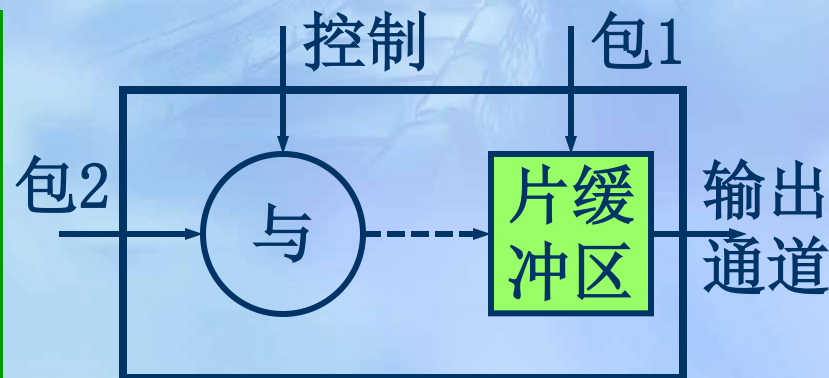
- ◆ 解决包冲突的策略
- ◆ 寻径算法
- ◆ 选播和广播寻径算法



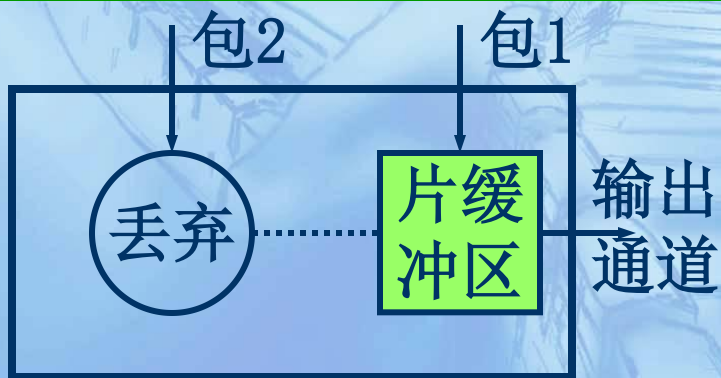
解决包冲突的策略



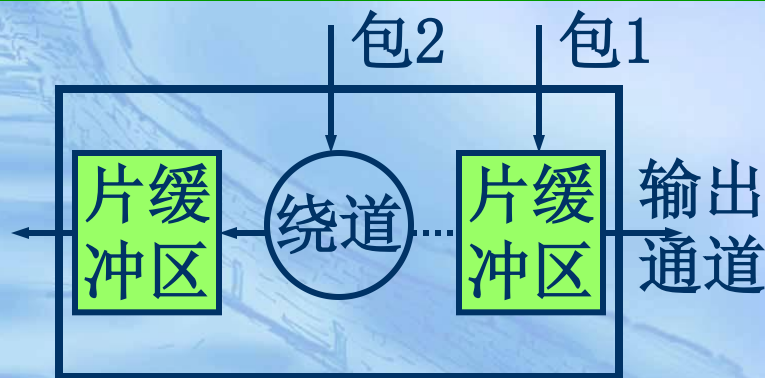
(a) 用缓冲实现虚拟通道



(b) 阻塞流控制



(c) 丢弃并重发



(d) 阻塞后绕道



寻径算法

寻径算法的目的是找出一条从源节点到目的节点的路径以便传送消息。寻径算法可分为两大类（都需要无死锁算法）

- ◆ 确定寻径算法

寻找的路径是预先唯一确定的，完全根据源和目的地址确定，与网络的状况无关。

- ◆ 自适应寻径算法

寻找的路径可能会有多条，取决于网络的状况。可采用虚拟通道避免死锁。



确定寻径算法

例如：**维序寻径**，是一种在多维网络中按照维序的特定顺序来选择后继通道。

维序寻径在二维网格网络中称为X-Y寻径，在三维网格网络中称为X-Y-Z寻径，...

维序寻径在超立方体（ n 维立方体）网络中称为E立方体寻径（E-cube routing）。



X-Y寻径

假定从任意源节点 $s=(x_1, y_1)$ 到任意目的节点 $d=(x_2, y_2)$ ，寻径从 s 开始，首先沿X方向前进到 d 所在的第 x_2 列，然后沿Y方向一直前进到 d 。

X-Y寻径共有四种寻址模式：**东-北、东-南、西-北、西-南**。例子见后图。



举例

东-北寻径模式:

$(2, 0) \rightarrow (3, 3)$

东-南寻径模式:

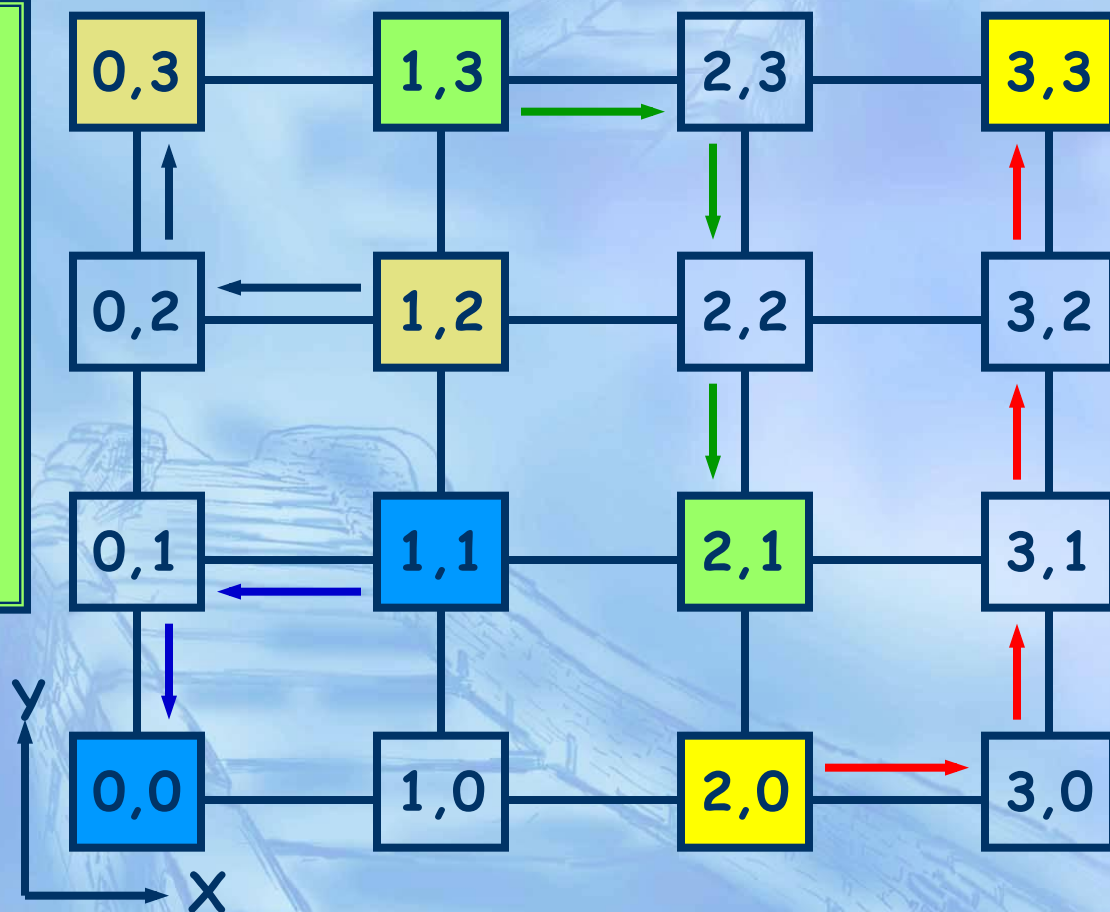
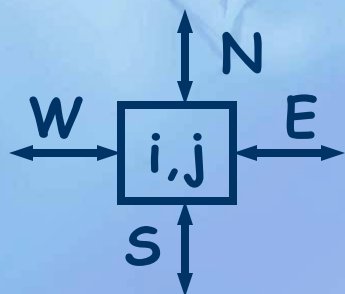
$(1, 3) \rightarrow (2, 1)$

西-北寻径模式:

$(1, 2) \rightarrow (0, 3)$

西-南寻径模式:

$(1, 1) \rightarrow (0, 0)$



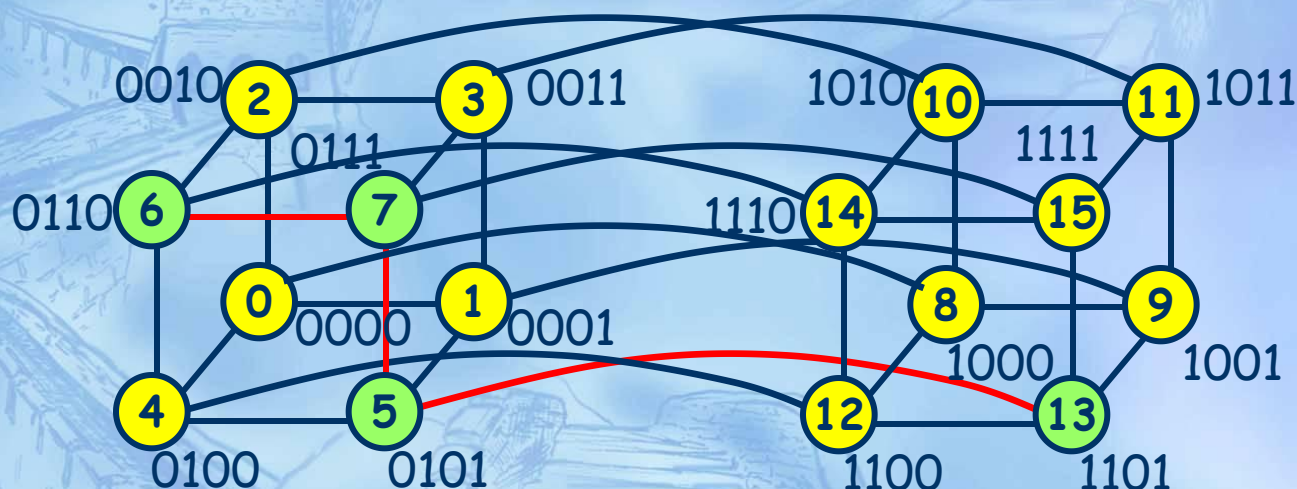


E立方体寻径

寻径是按照维的顺序进行的（维1→维2→...→维n）。如果源节点和目的节点二进制编号的第i位相同，则沿维i方向不需要寻径，否则从当前节点沿着这一维方向走到其他节点，重复此过程直至到达目的节点。



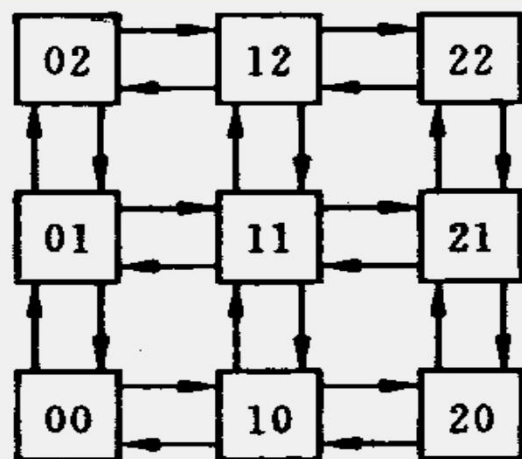
举例



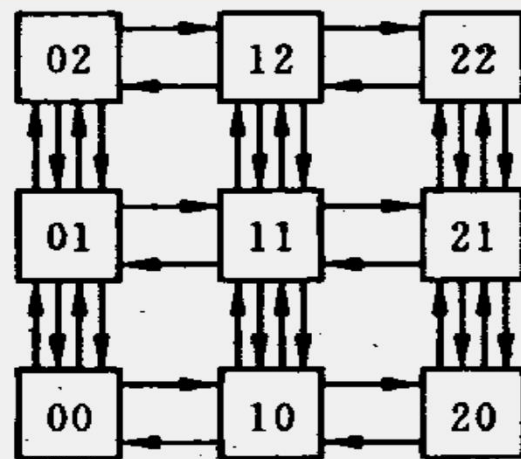
源 结 点 : $s=0110$

目的节点 : $d=1101$

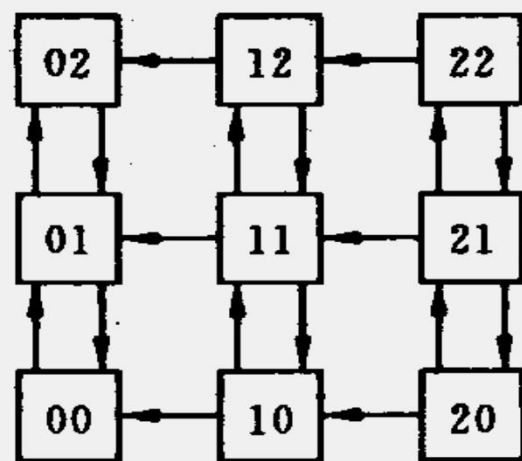
路 径 : $0110 \rightarrow 0111 \rightarrow 0101 \rightarrow 1101$



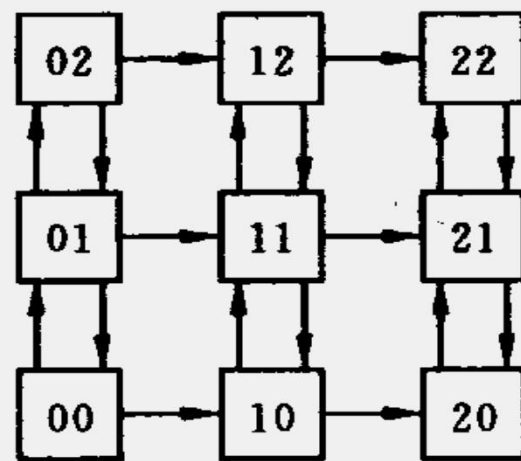
(a) 没有虚拟通道的
原形网络



(b) Y维方向有两对
虚拟通道



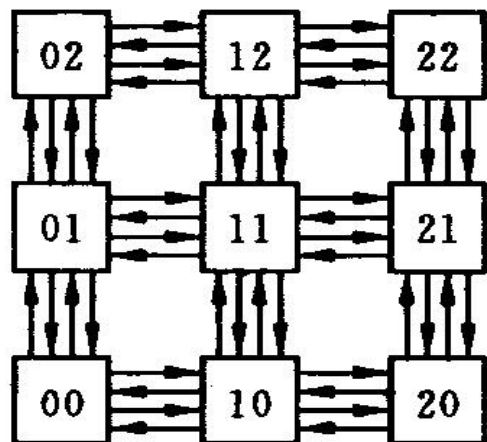
(c) 向西方向传输消息



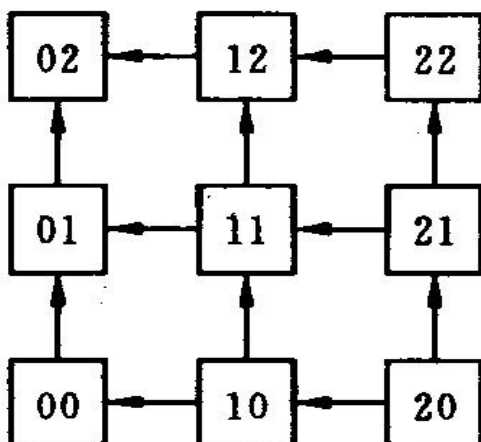
(d) 向东方向传输消息

利用虚拟通道避免死锁的自适应X-Y寻径
(在Y维上用了2对虚拟通道)

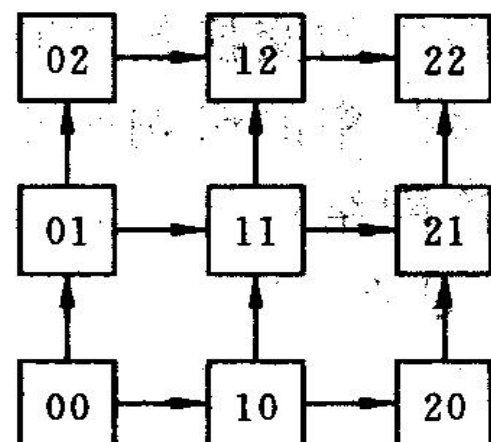
利用虚拟通道避免死锁的自适应X-Y寻径 (在X和Y维上各用了2对虚拟通道)



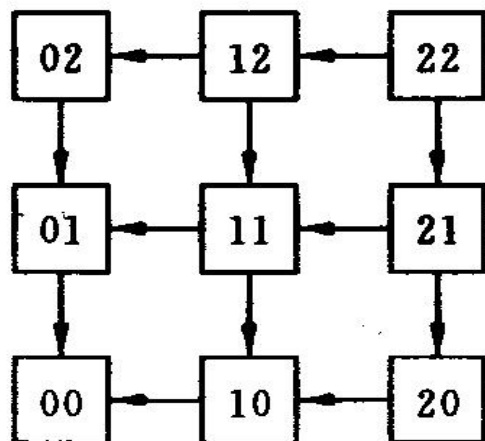
(a) 双通道的 3×3 网络



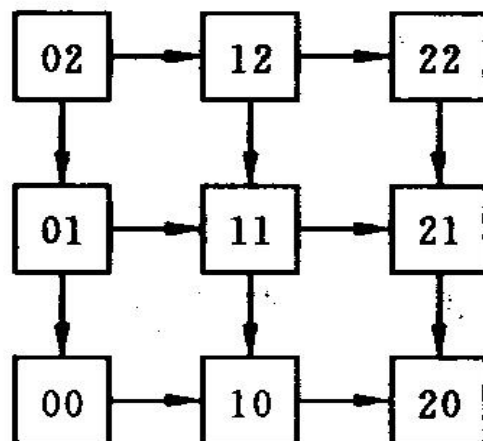
(b) 西-北子网



(c) 东-北子网



(d) 西-南子网



(e) 东-南子网



通信模式

多计算机网络中会出现四种通信模式:

- ◆ **单播 (unicast) 模式**
一对一的通信方式。
- ◆ **选播 (multicast) 模式**
一对多的通信方式。
- ◆ **广播 (broadcast) 模式**
一对全体的通信方式。
- ◆ **会议 (conference) 模式**
多对多的通信方式。



通信效率

描述通信效率常用有两个参数：

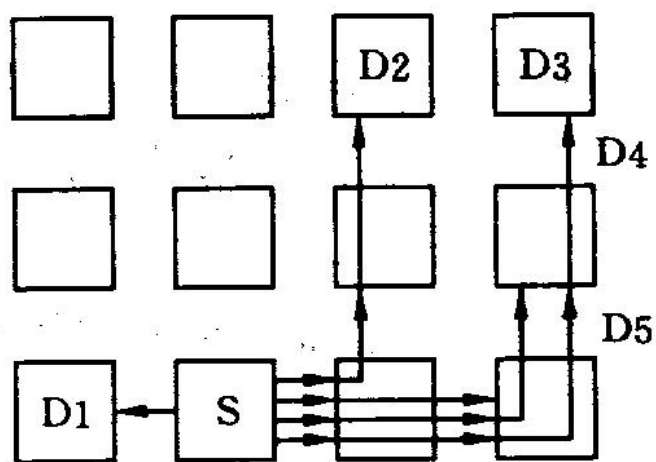
- ◆ **通道流量**

可用传输有关信息所使用的通道数来表示。

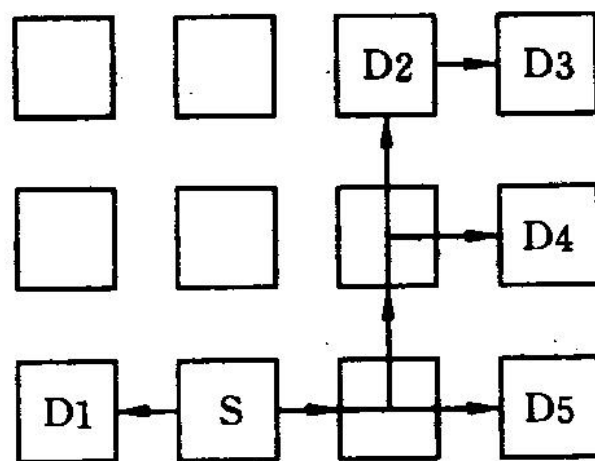
- ◆ **通信时延**

用包的最长传输时间来表示。

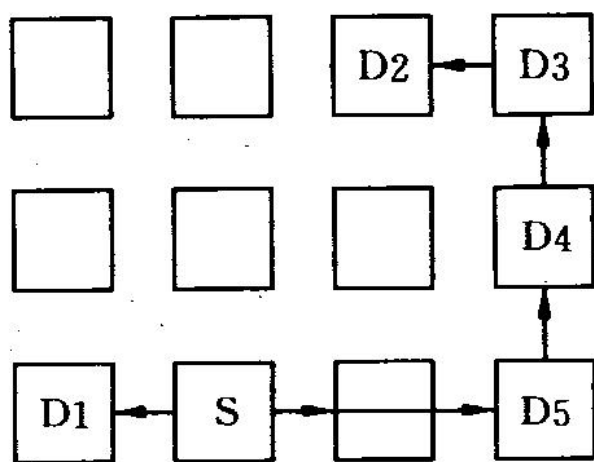
提示：优化的寻径网络应能以最小流量和最小时延实现有关通信模式。



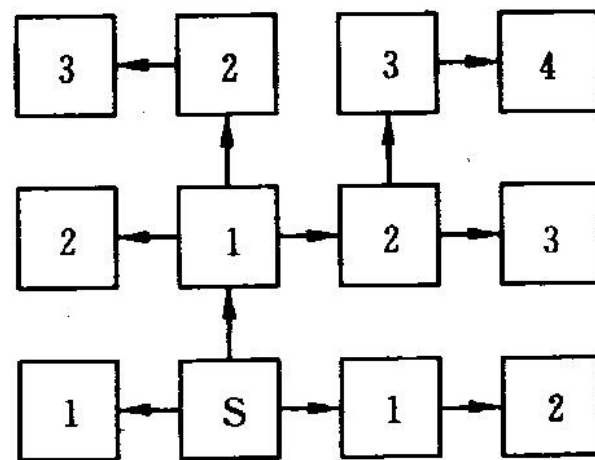
(a) 5次单播，流量等于13，距离等于4



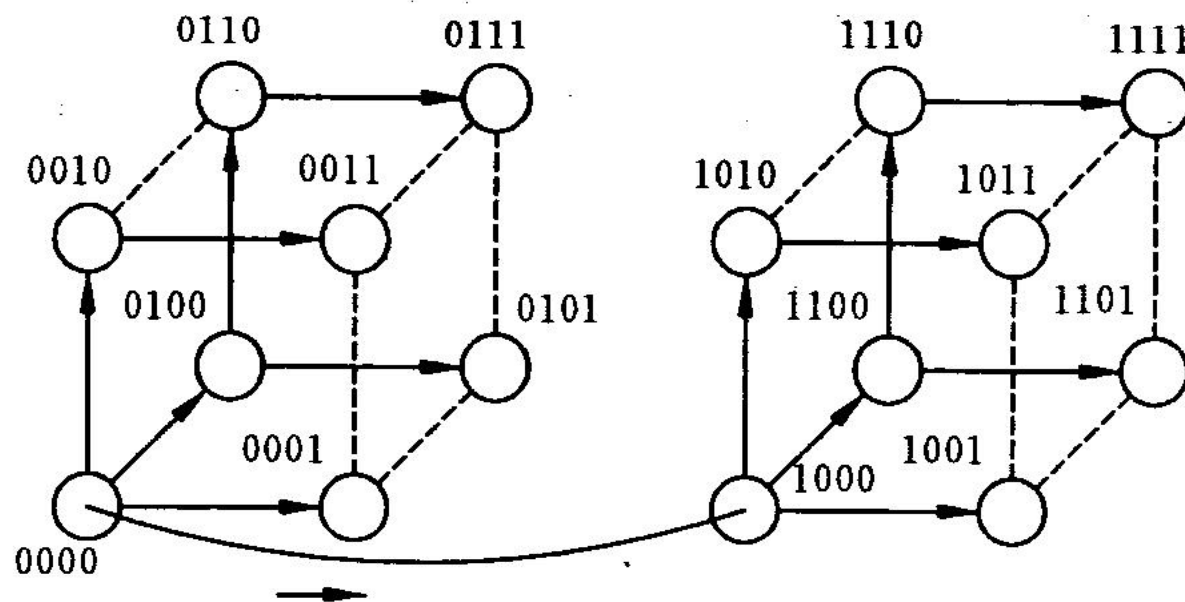
(b) 一种流量等于7、距离等于4的选播模式



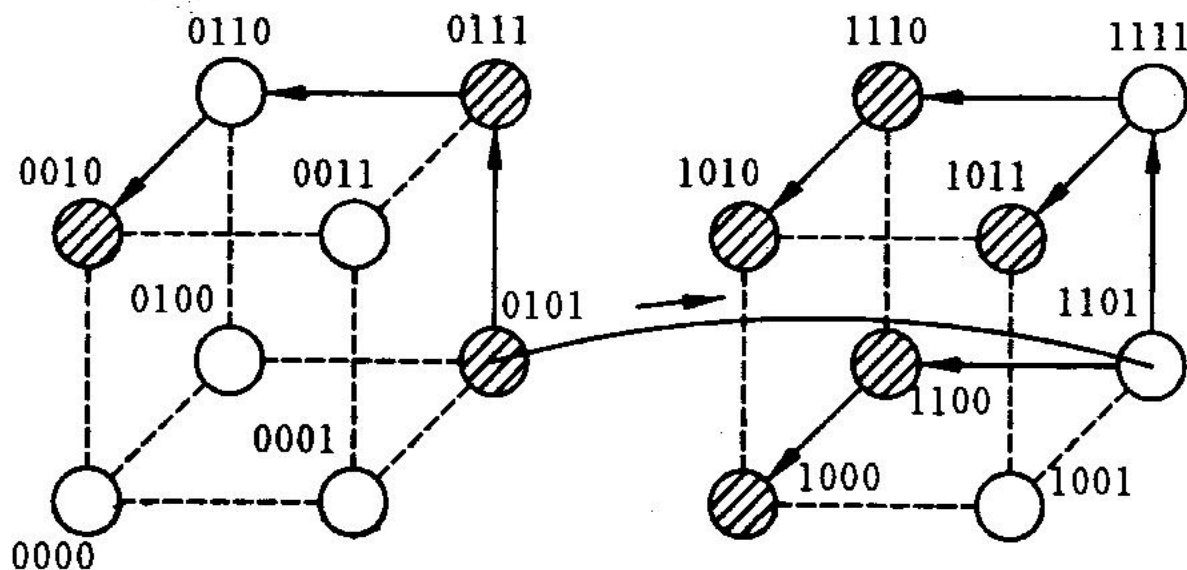
c) 另一种流量等于6、距离等于5的选播模式



(d) 通过树结构广播给所有结点
(结点中的数字表示树的层次号)



(a) 根结点为0000的4立方体的广播树



(b) 从结点0101到7个目的结点1100、0111、1010、1110、1011、1000和0010的选播树